DENNIS GUNAWAN



IF351 DATABASE SYSTEMS

07 SELECT STATEMENT 2 & 3

REVIEW:SELECT STATEMENT 1

SELECT

FROM

WHERE

Operators

Aggregation Functions

COURSE SUB LEARNING OUTCOMES (SUB-CLO)

Sub-CLO 7

Students are able to use SELECT statement (C3)

OUTLINE

GROUP BY

HAVING

ORDER BY

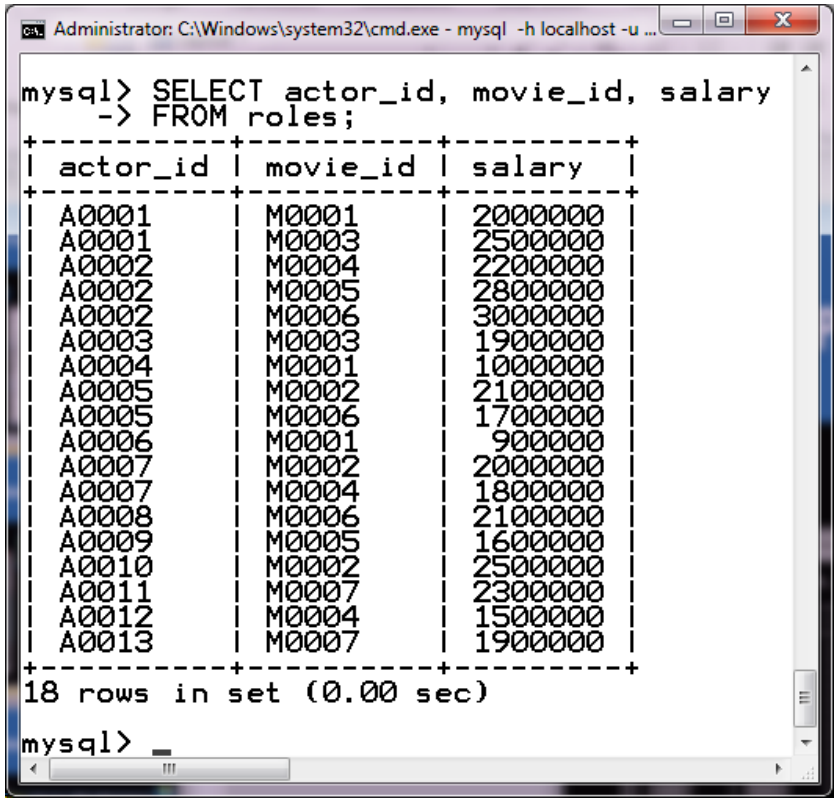
LIMIT

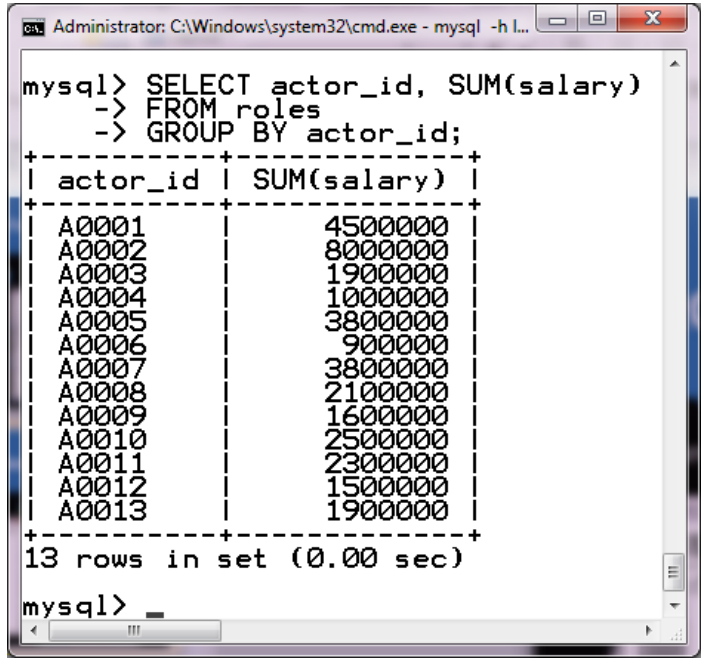
CASE Expression

Subquery

GROUP BY CLAUSE

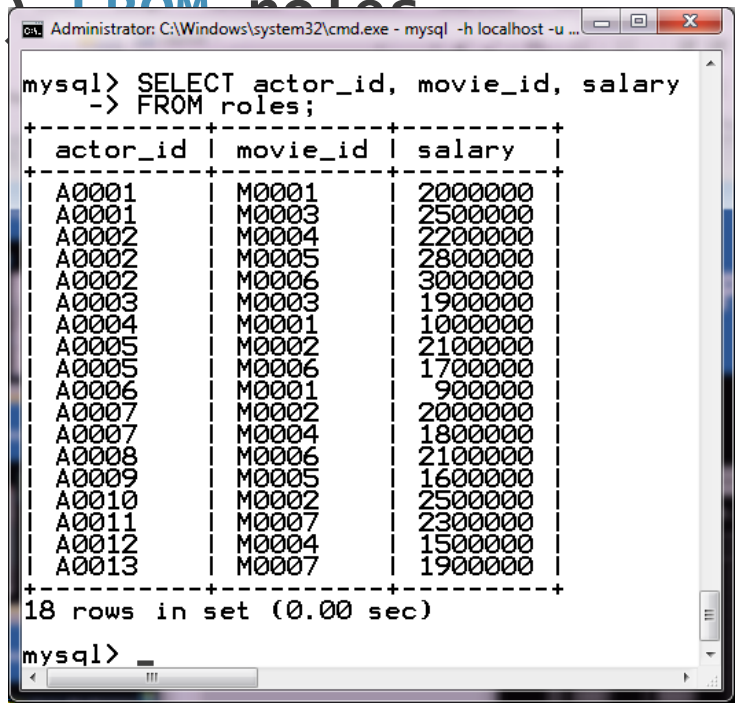
```
SELECT actor_id, SUM(salary)
FROM roles
GROUP BY actor_id;
```

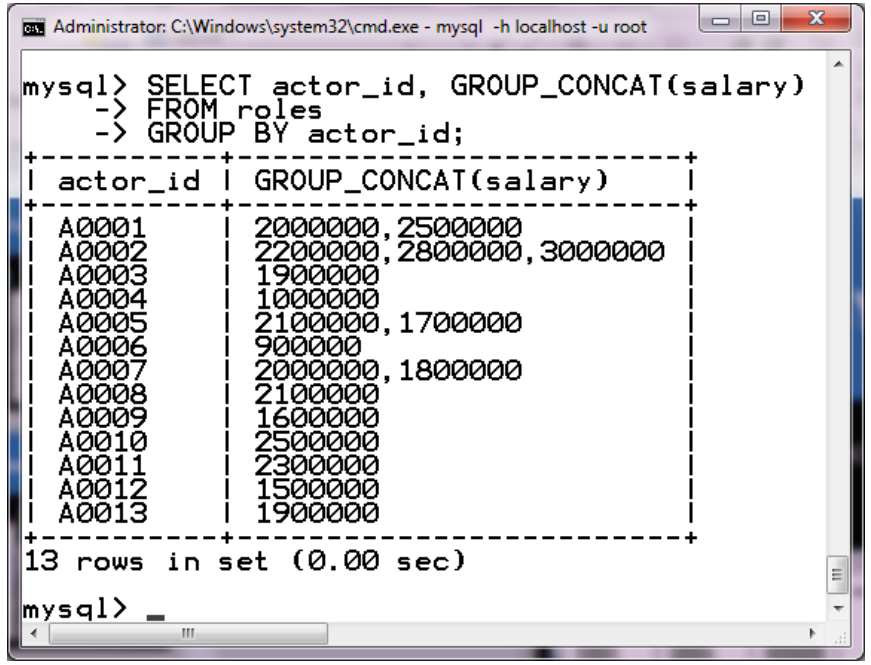




GROUP BY CLAUSE:GROUP_CONCAT FUNCTION

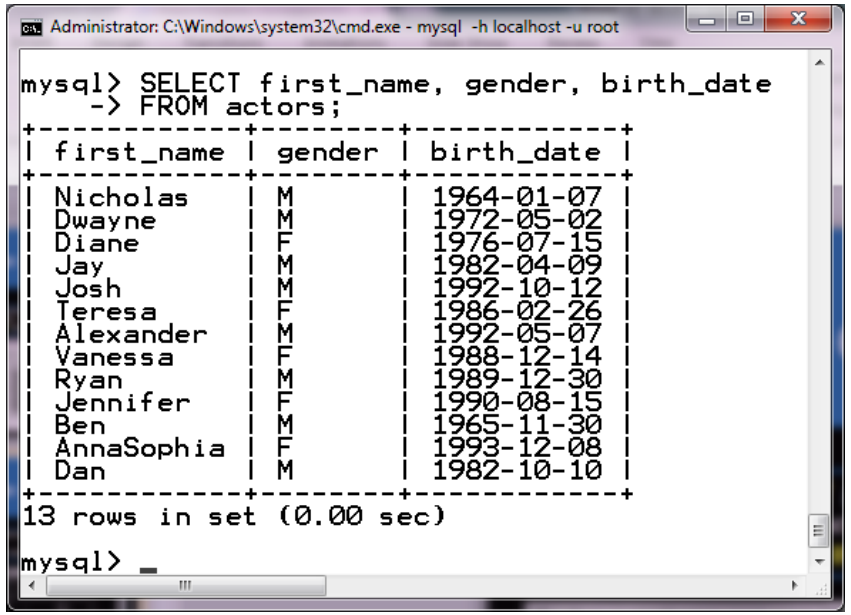
```
SELECT actor_id,
GROUP_CONCAT(salary) FROM roles
GROUP BY actor_id;
```

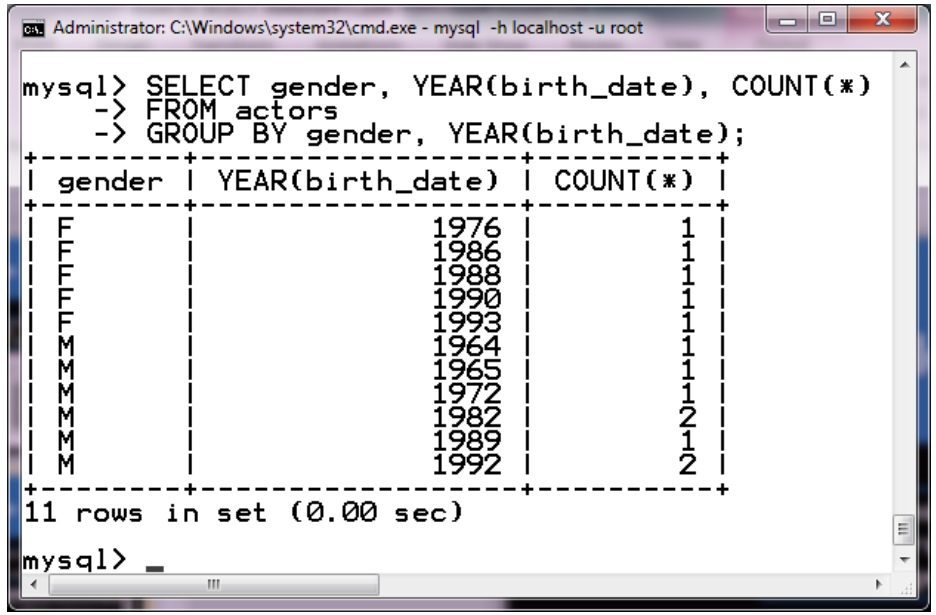




GROUP BY CLAUSE

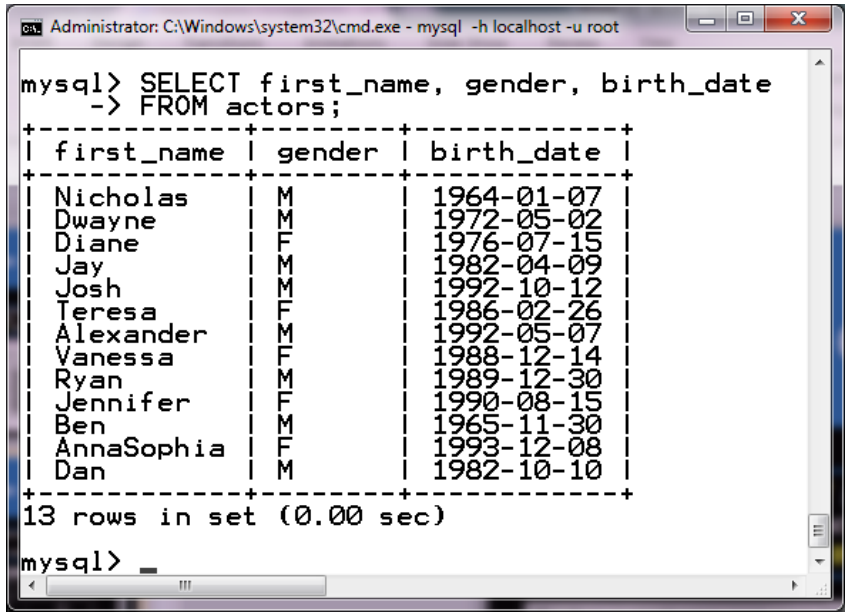
```
SELECT gender, YEAR(birth_date), COUNT(*)
FROM actors
GROUP BY gender, YEAR(birth_date);
```

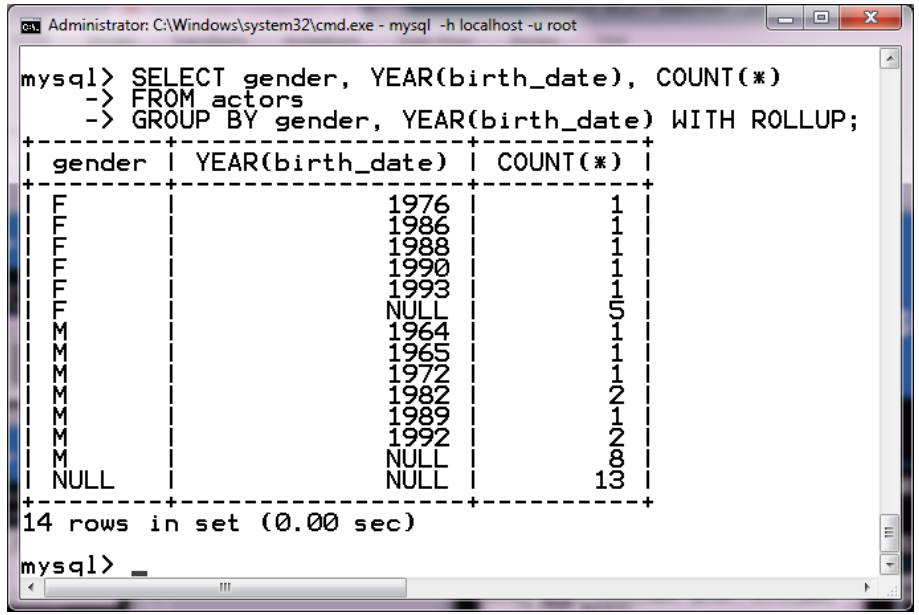




GROUP BY CLAUSE:WITH ROLLUP

```
SELECT gender, YEAR(birth_date), COUNT(*)
FROM actors
GROUP BY gender, YEAR(birth_date) WITH ROLLUP;
```





HAVING CLAUSE

HAVING Clause

HAVING clause is used to select rows after a GROUP BY clause has been executed

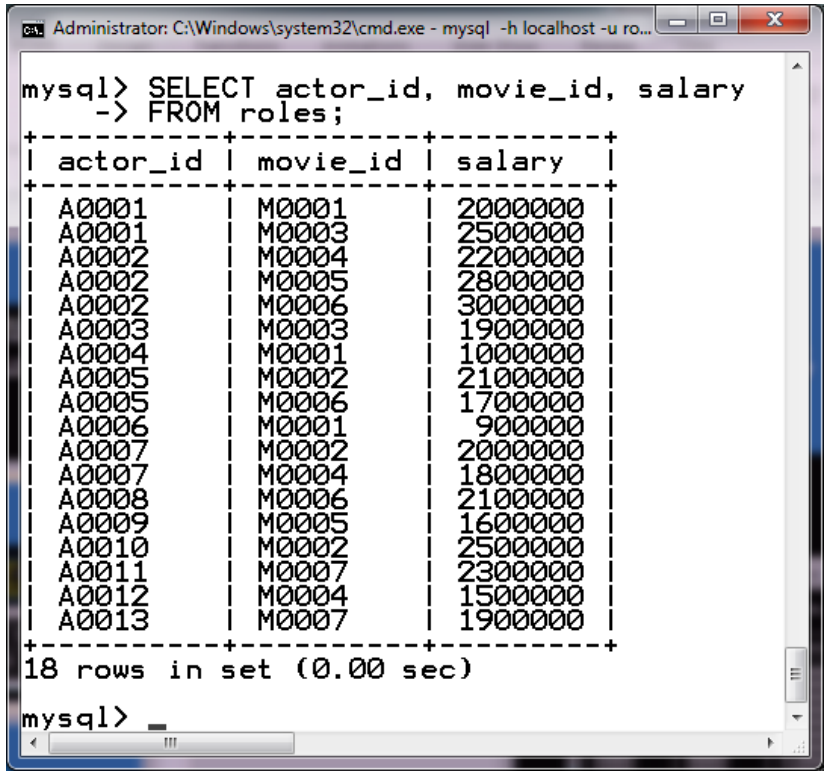
Expressions in the condition of a HAVING clause can contain aggregation functions

WHERE Clause

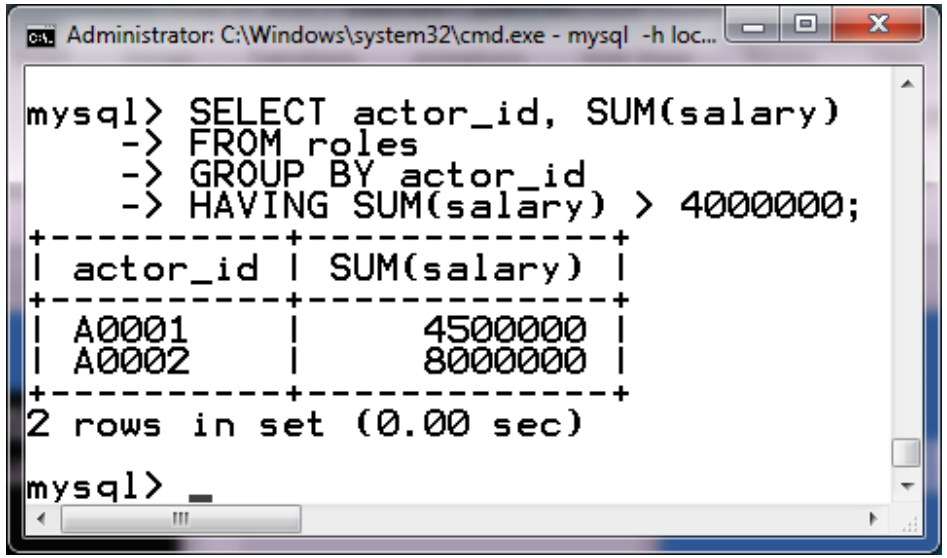
WHERE clause is used to select rows after the FROM clause has been processed

Expressions in the condition of a WHERE clause cannot contain aggregation functions

HAVING CLAUSE

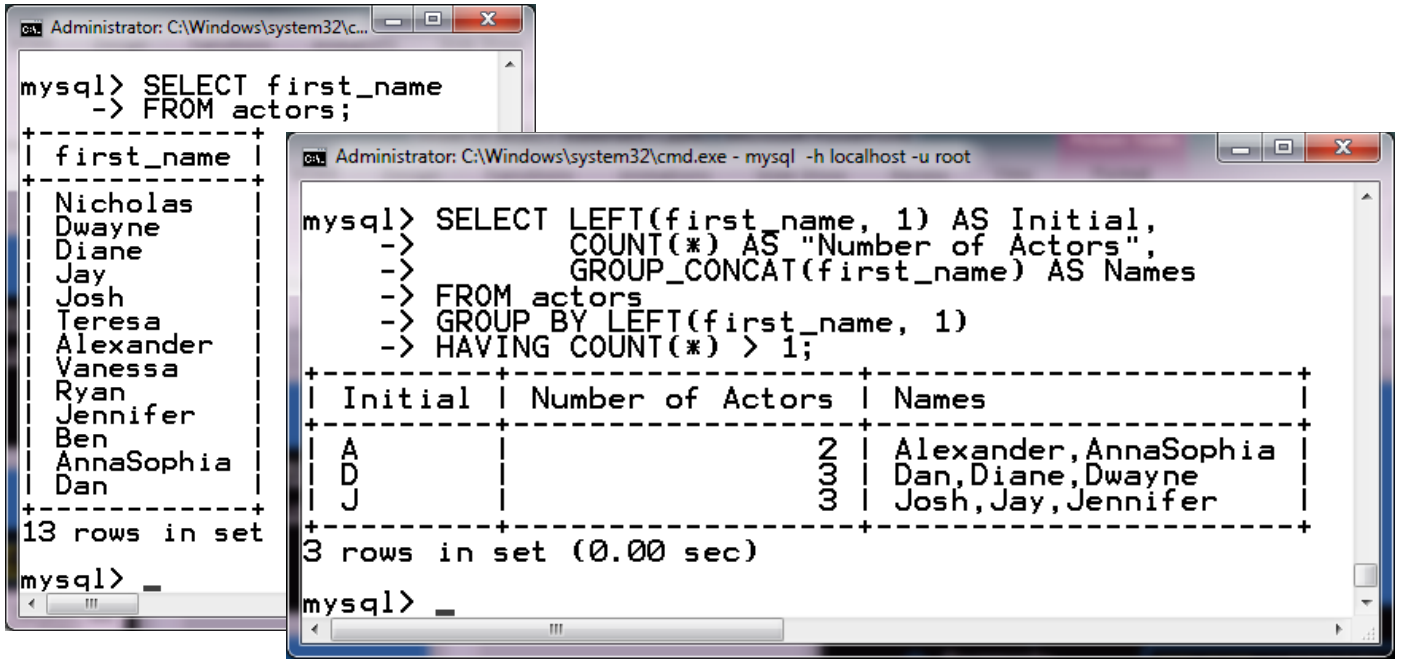



```
SELECT actor_id, SUM(salary)
FROM roles
GROUP BY actor_id
HAVING SUM(salary) > 4000000;
```



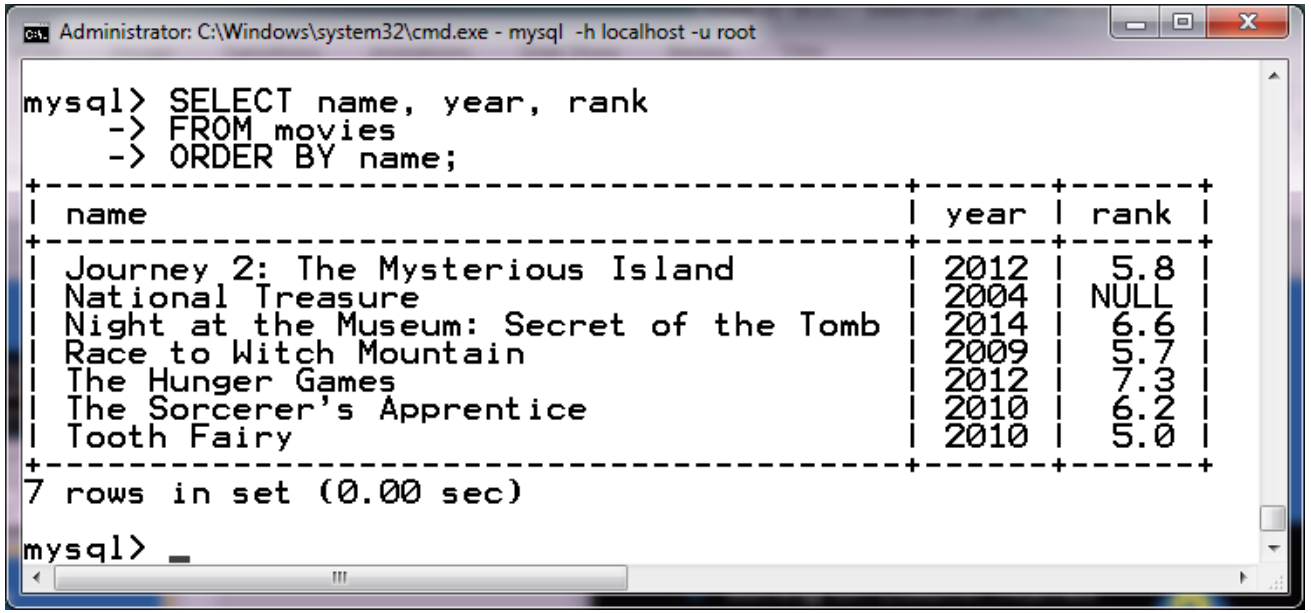
HAVING CLAUSE

```
SELECT LEFT(first_name, 1) AS Initial,
       COUNT(*) AS “Number of Actors”,
       GROUP_CONCAT(first_name) AS Names
FROM actors
GROUP BY LEFT(first_name, 1)
HAVING COUNT(*) > 1;
```



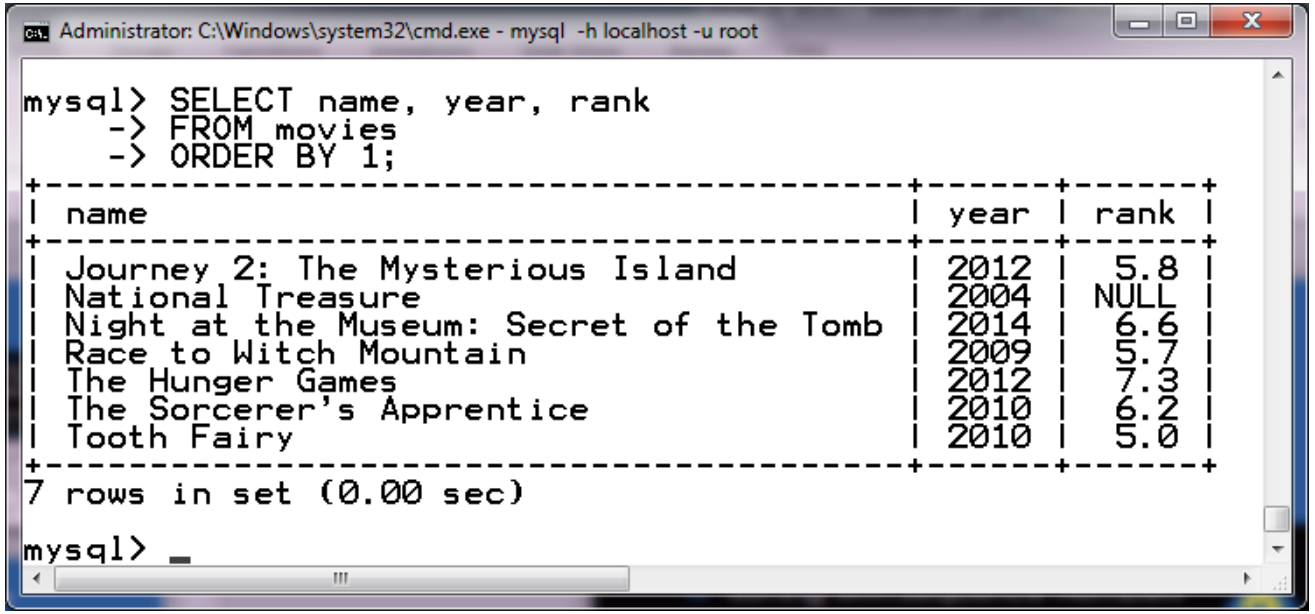
ORDER BY CLAUSE:SORTING ON COLUMN NAMES

```
SELECT name, year, rank
FROM movies
ORDER BY name;
```



ORDER BY CLAUSE: SORTING WITH SEQUENCE NUMBERS

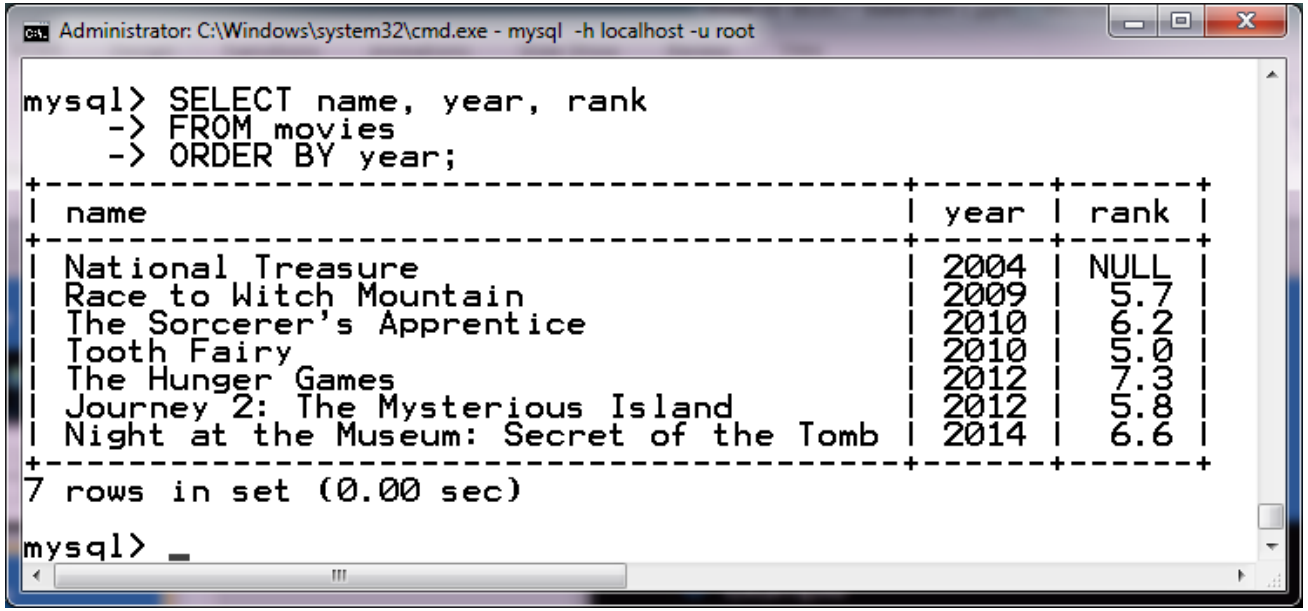
```
SELECT name, year, rank
FROM movies
ORDER BY 1;
```



ORDER BY CLAUSE

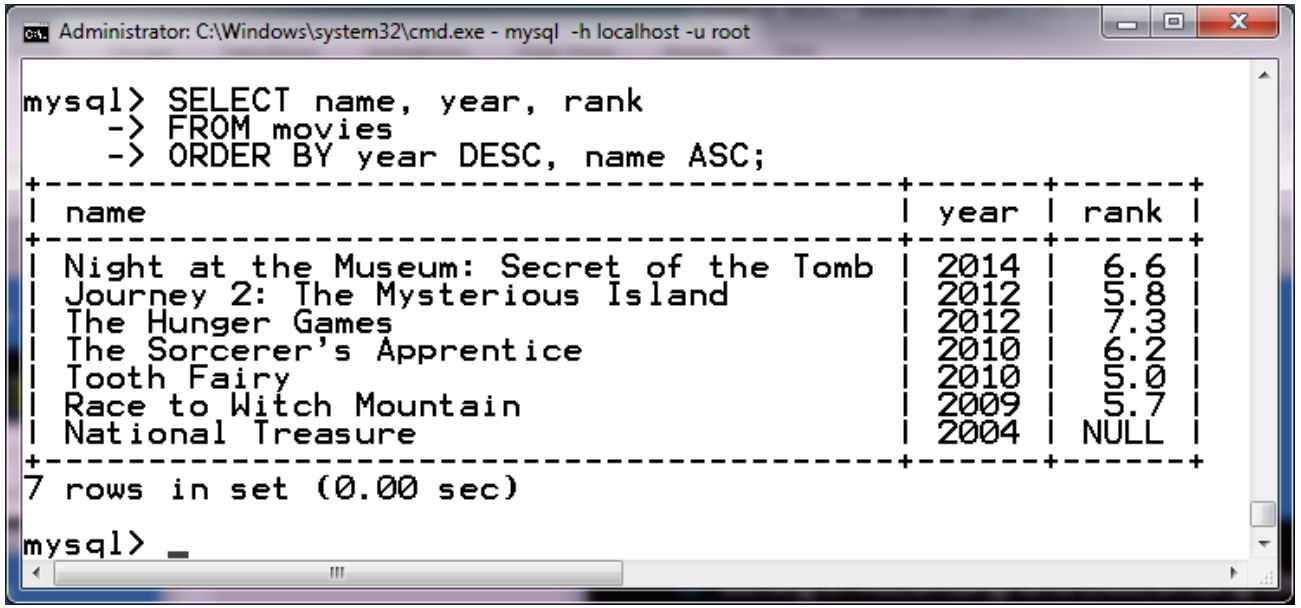
```
SELECT name, year, rank
FROM movies
ORDER BY year;
```

```
SELECT name, year, rank
FROM movies
ORDER BY 2;
```



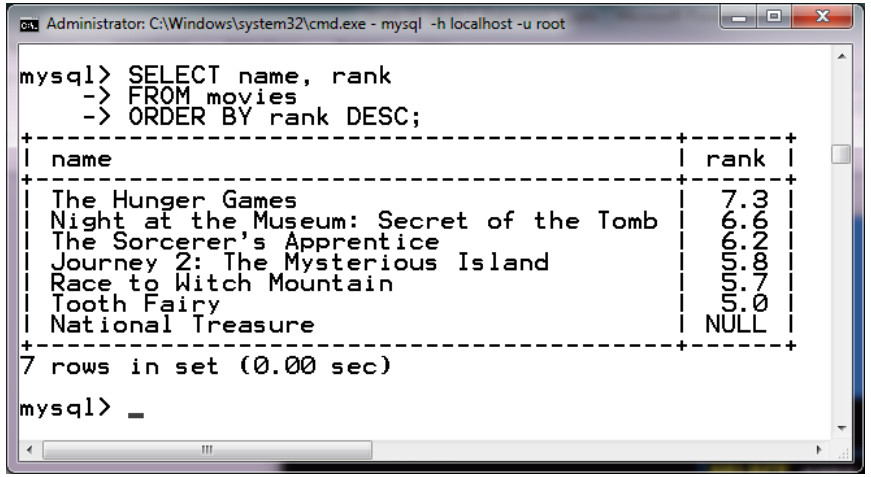
ORDER BY CLAUSE: SORTING IN ASCENDING AND DESCENDING ORDER

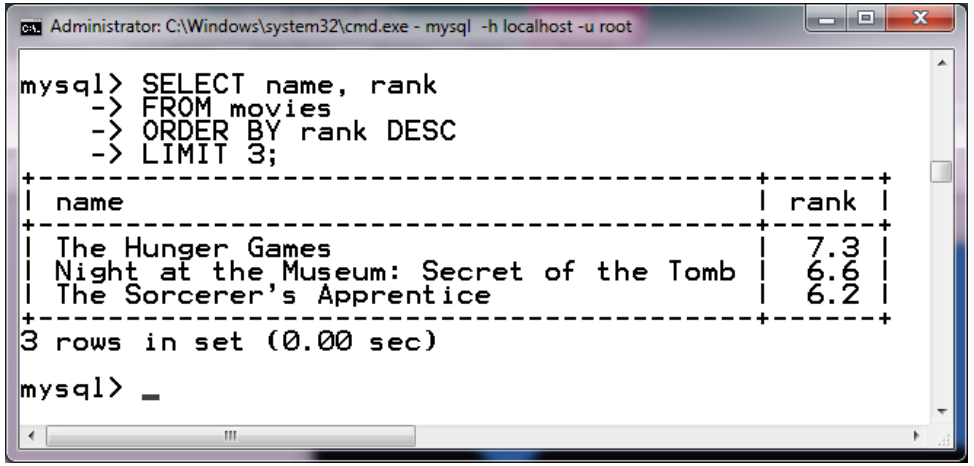
```
SELECT name, year, rank
FROM movies
ORDER BY year DESC, name ASC;
```



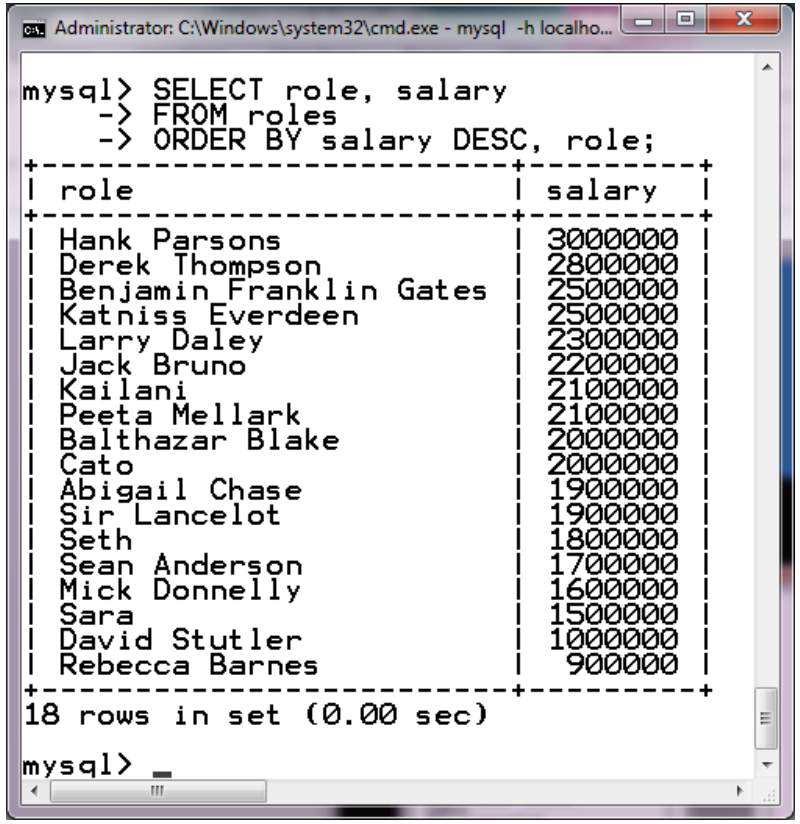
LIMIT CLAUSE

```
SELECT name, rank
FROM movies
ORDER BY rank DESC
LIMIT 3;
```

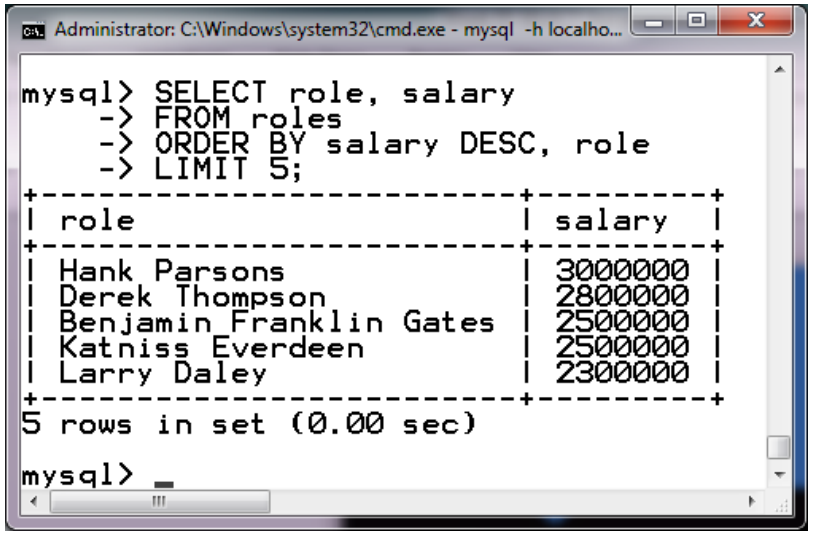




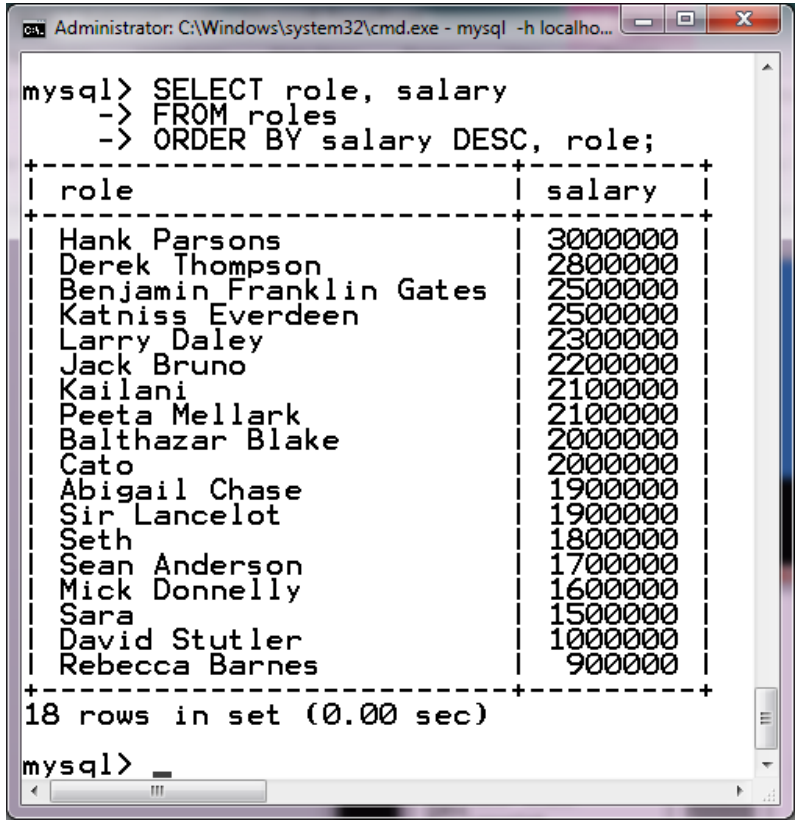
LIMIT CLAUSE



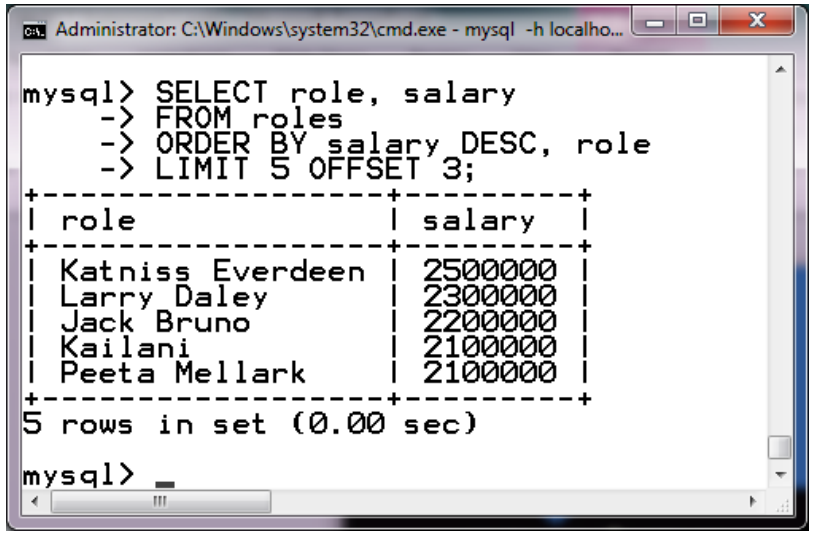

```
SELECT role, salary
FROM roles
ORDER BY salary DESC, role
LIMIT 5;
```



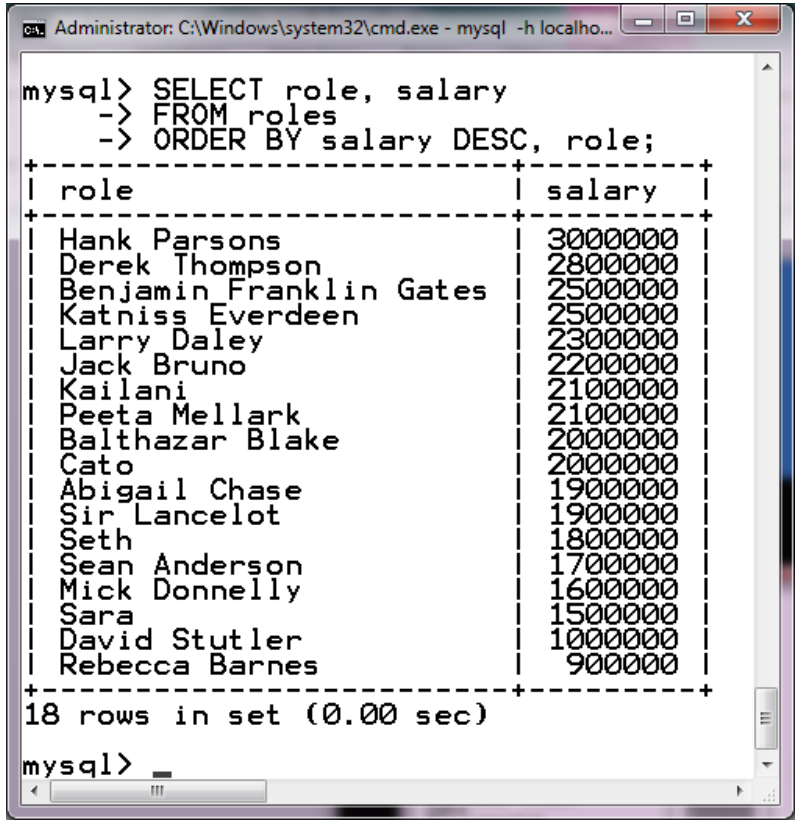
LIMIT CLAUSE: LIMIT WITH AN OFFSET



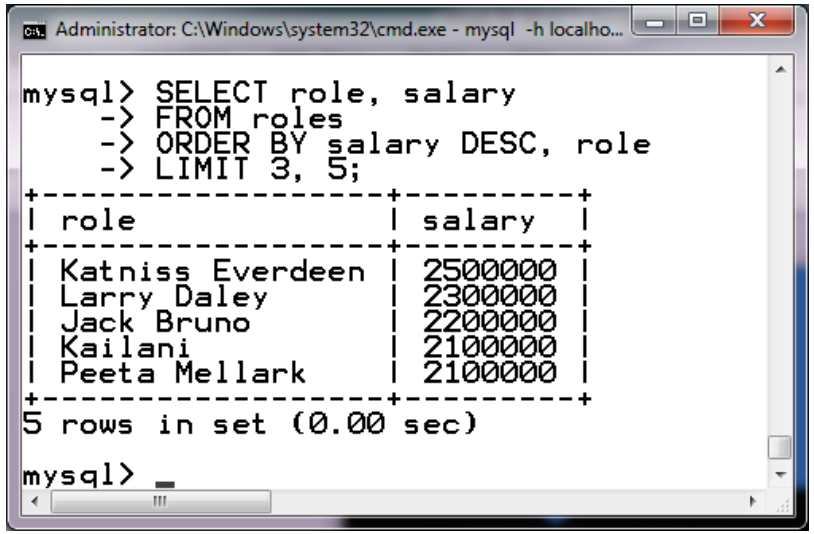

```
SELECT role, salary
FROM roles
ORDER BY salary DESC, role
LIMIT 5 OFFSET 3;
```



LIMIT CLAUSE: LIMIT WITH AN OFFSET

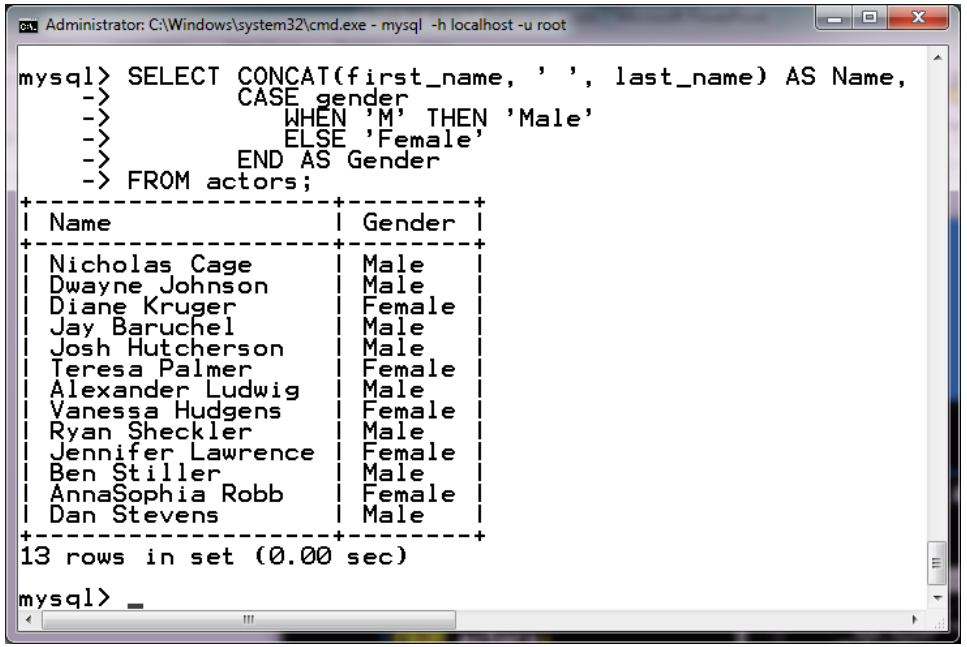



```
SELECT role, salary
FROM roles
ORDER BY salary DESC, role
LIMIT 3, 5;
```



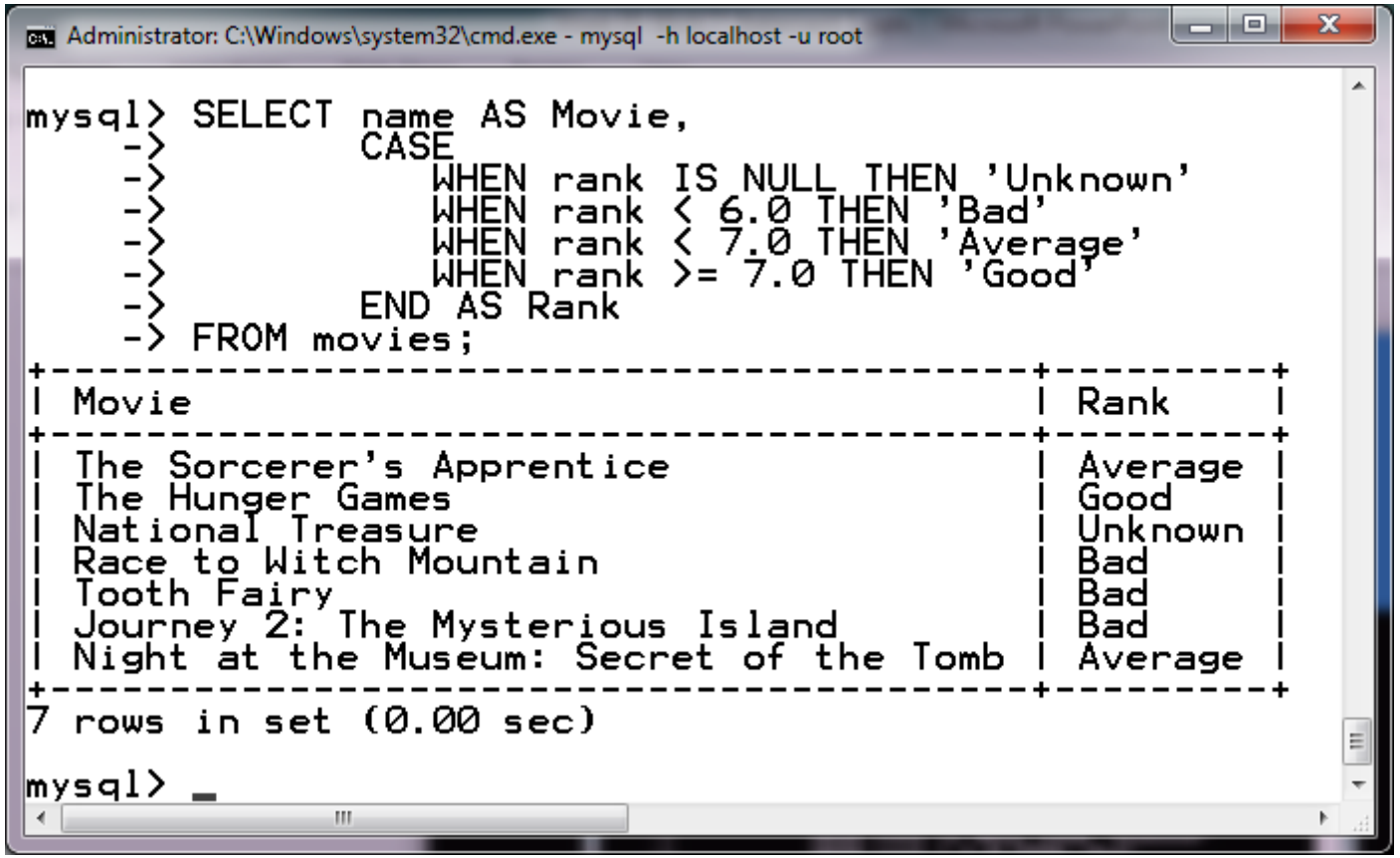
CASE EXPRESSION

```
SELECT CONCAT(first_name, ‘ ’, last_name) AS Name,
       CASE gender
          WHEN ‘M’ THEN ‘Male’
          ELSE ‘Female’
       END AS Gender
FROM actors;
```



CASE EXPRESSION

```
SELECT name AS Movie,
       CASE
          WHEN rank IS NULL THEN ‘Unknown’
          WHEN rank < 6.0 THEN ‘Bad’
          WHEN rank < 7.0 THEN ‘Average’
          WHEN rank >= 7.0 THEN ‘Good’
       END AS Rank
FROM movies;
```



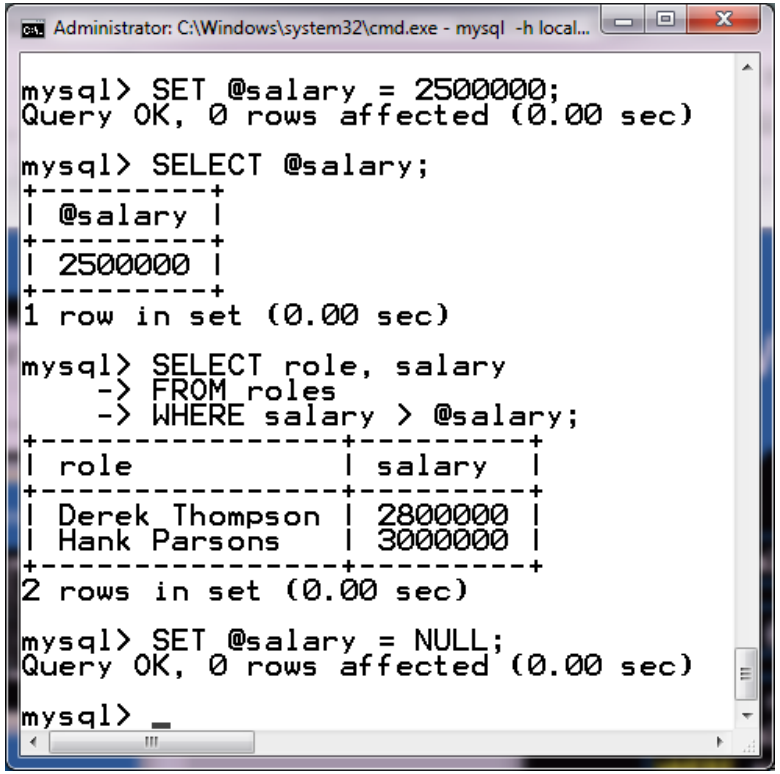
USER VARIABLE & SET STATEMENT

```
SET @salary = 2500000;

SELECT @salary;

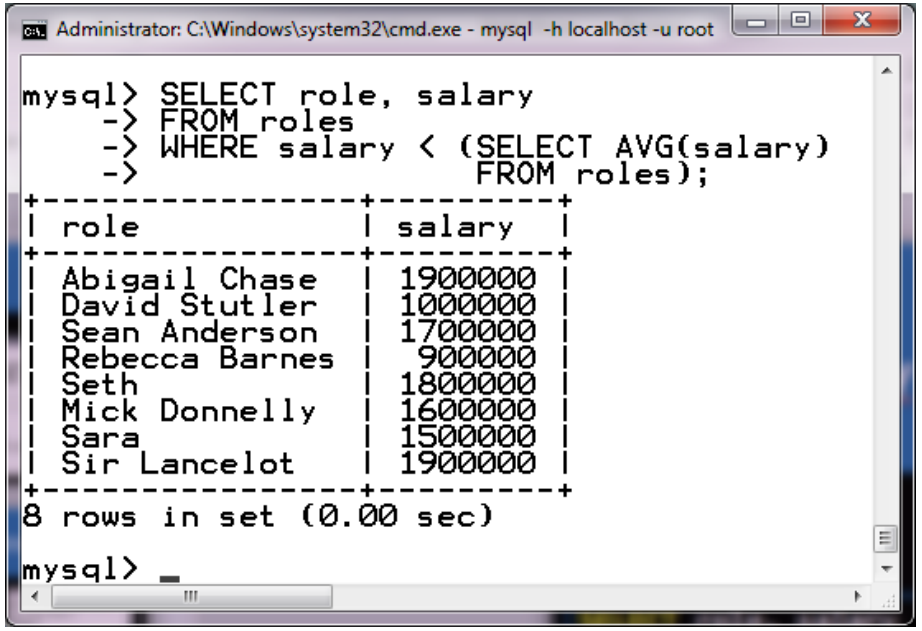
SELECT role, salary
FROM roles
WHERE salary > @salary;

SET @salary = NULL;
```



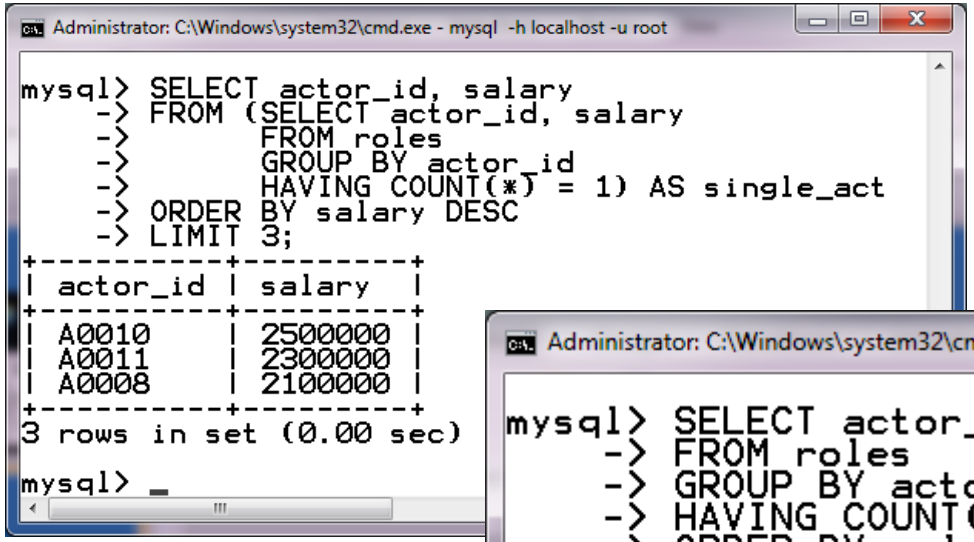
SUBQUERY

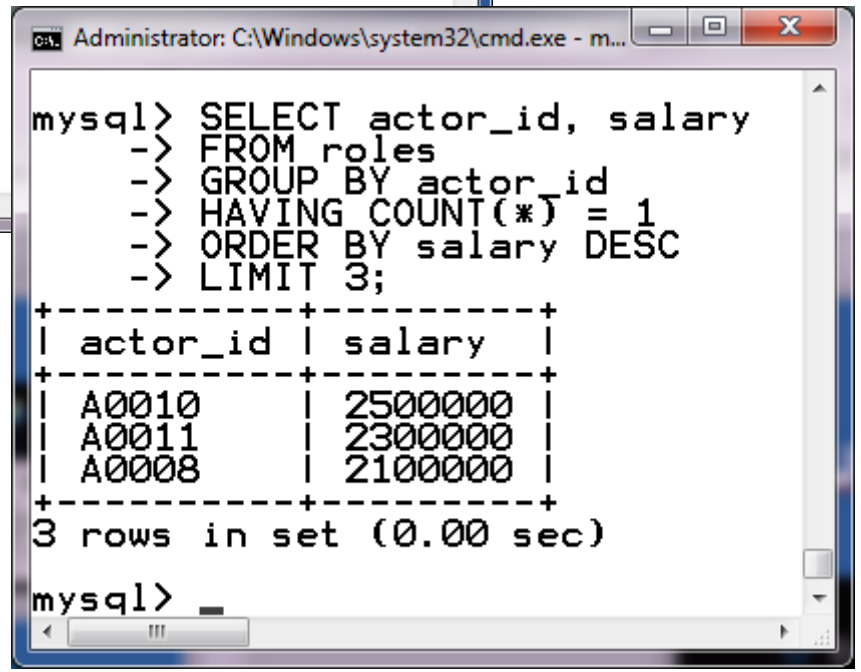
```
SELECT role, salary
FROM roles
WHERE salary < (SELECT AVG(salary)
                FROM roles);
```



SUBQUERY

```
SELECT actor_id, salary
FROM (SELECT actor_id, salary
      FROM roles
      GROUP BY actor_id
      HAVING COUNT(*) = 1) AS single_act
ORDER BY salary DESC
LIMIT 3;
```

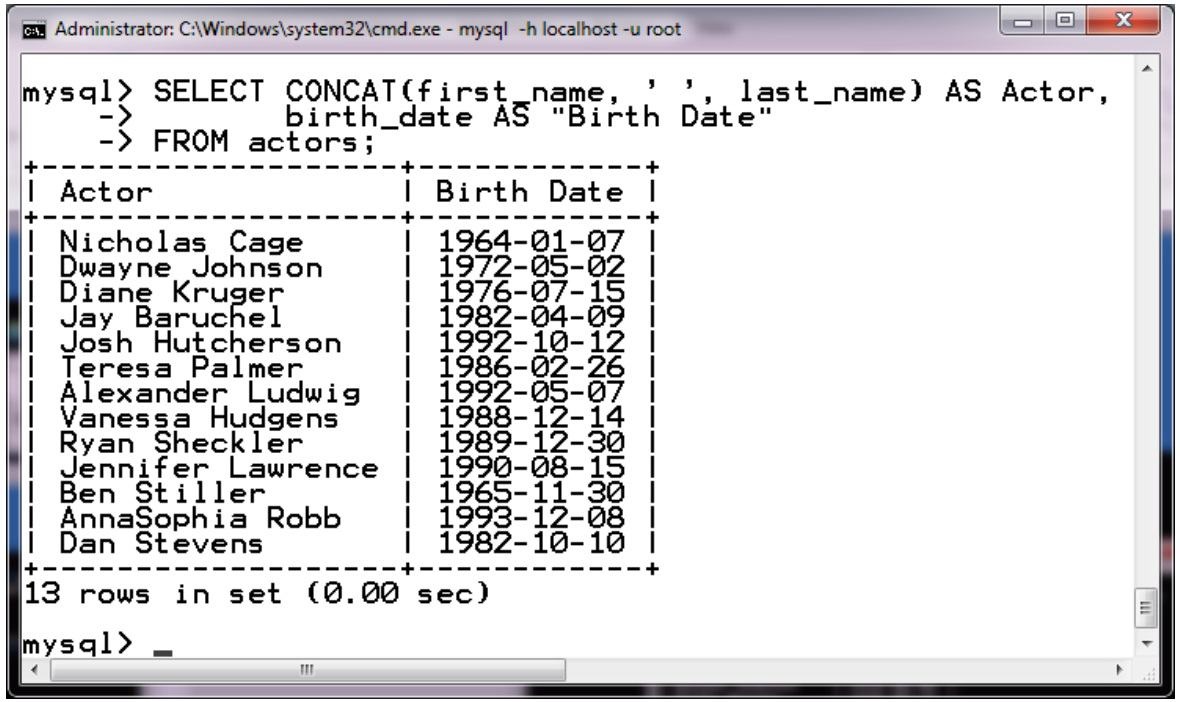





```
SELECT actor_id, salary
FROM roles
GROUP BY actor_id
HAVING COUNT(*) = 1
ORDER BY salary DESC
LIMIT 3;
```

SUBQUERY:ALL & ANY OPERATORS

```
SELECT CONCAT(first_name, ‘ ’, last_name) AS Actor,
       birth_date AS “Birth Date”
FROM actors
WHERE birth_date < ALL
      (SELECT birth_date
      FROM actors
       WHERE YEAR(birth_date) % 2 <> 0);
```



SUBQUERY:ALL & ANY OPERATORS

Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root

```
mysql> SELECT CONCAT(first_name, ' ', last_name) AS Actor,
    ->        birth_date AS "Birth Date"
    -> FROM actors;
+--------------------+------------+
| Actor              | Birth Date |
+--------------------+------------+
| Nicholas Cage      | 1964-01-07 |
| Dwayne Johnson     | 1972-05-02 |
| Diane Kruger       | 1976-07-15 |
| Jay Baruchel       | 1982-04-09 |
| Josh Hutcherson    | 1992-10-12 |
| Teresa Palmer      | 1986-02-26 |
| Alexander Ludwig   | 1992-05-07 |
| Vanessa Hudgens    | 1988-12-14 |
| Ryan Sheckler      | 1989-12-30 |
| Jennifer Lawrence  | 1990-08-15 |
| Ben Stiller        | 1965-11-30 |
| AnnaSophia Robb    | 1993-12-08 |
| Dan Stevens        | 1982-10-10 |
+--------------------+------------+
13 rows in set (0.00 sec)

mysql> _
```

< ALL → < MINIMUM

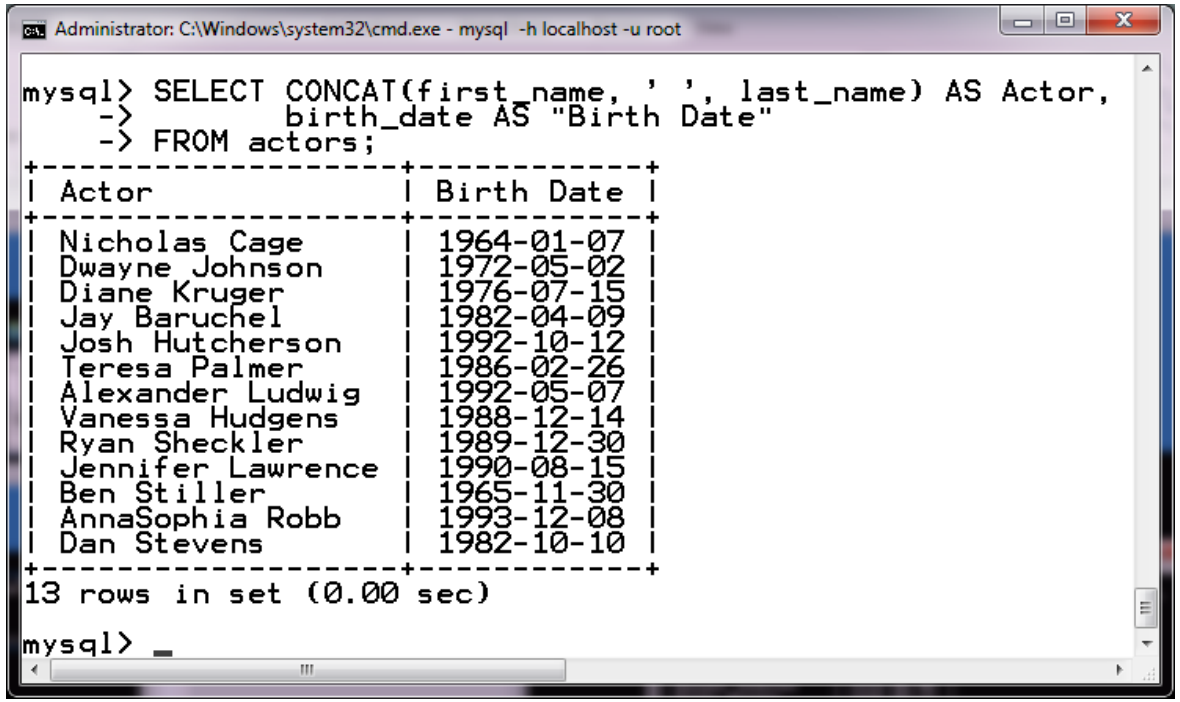
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root

```
mysql> SELECT CONCAT(first_name, ' ', last_name) AS Actor,
    ->        birth_date AS "Birth Date"
    -> FROM actors
    -> WHERE birth_date < ALL
    ->       (SELECT birth_date
    ->        FROM actors
    ->        WHERE YEAR(birth_date) % 2 <> 0);
+---------------+------------+
| Actor         | Birth Date |
+---------------+------------+
| Nicholas Cage | 1964-01-07 |
+---------------+------------+
1 row in set (0.00 sec)

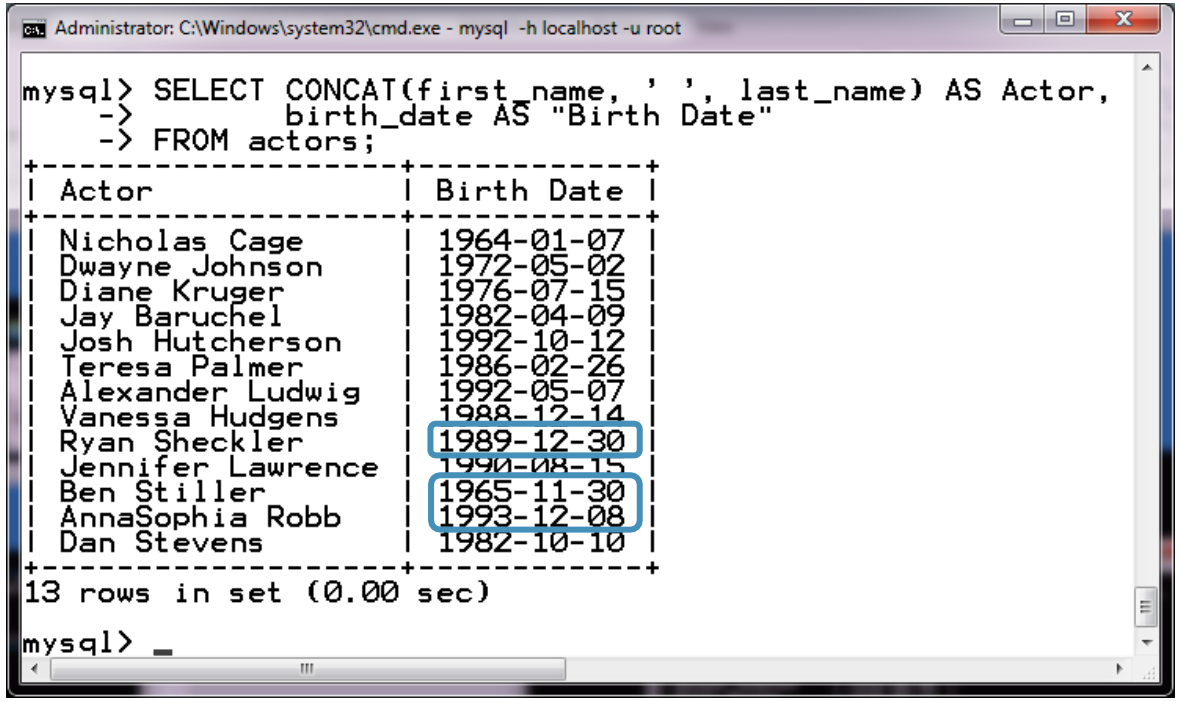
mysql> _
```

SUBQUERY:ALL & ANY OPERATORS

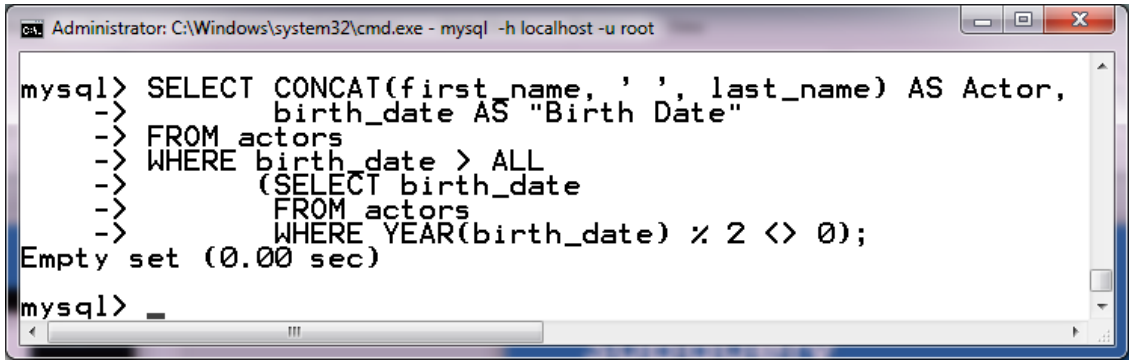
```
SELECT CONCAT(first_name, ‘ ’, last_name) AS Actor,
       birth_date AS “Birth Date”
FROM actors
WHERE birth_date > ALL
      (SELECT birth_date
      FROM actors
       WHERE YEAR(birth_date) % 2 <> 0);
```



SUBQUERY:ALL &ANY OPERATORS

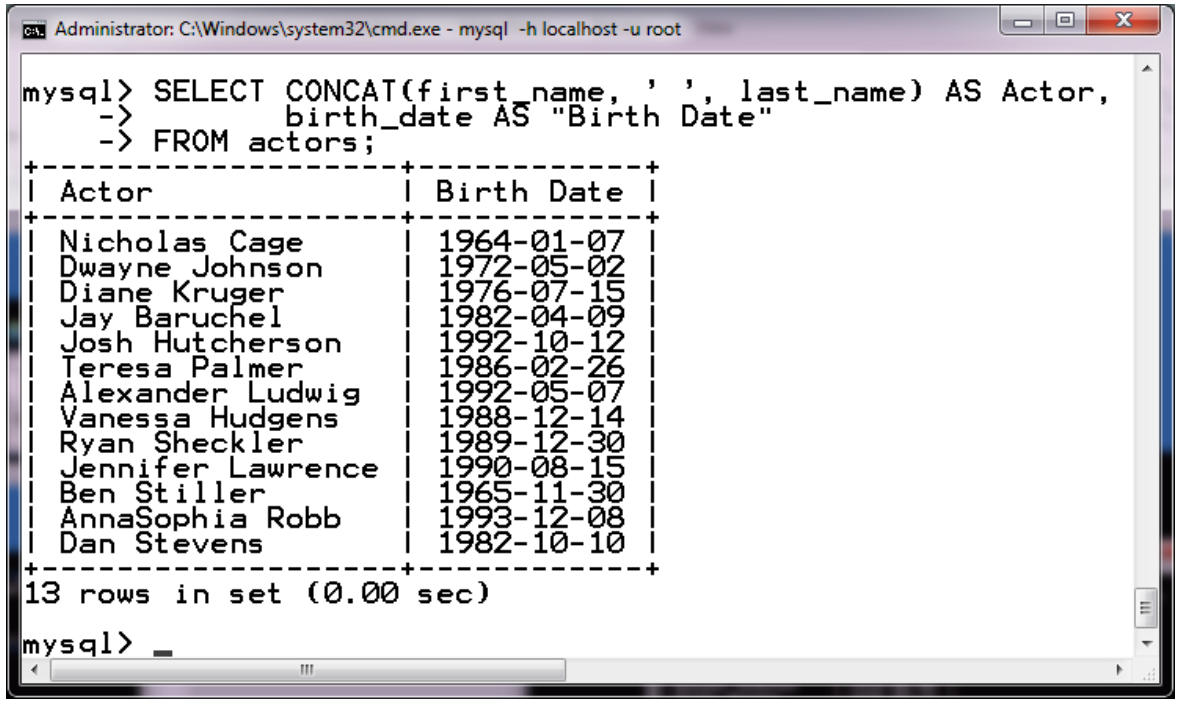


> ALL → > MAXIMUM



SUBQUERY:ALL & ANY OPERATORS

```
SELECT CONCAT(first_name, ‘ ’, last_name) AS Actor,
       birth_date AS “Birth Date”
FROM actors
WHERE birth_date < ANY
      (SELECT birth_date
      FROM actors
       WHERE YEAR(birth_date) % 2 <> 0);
```



SUBQUERY:ALL & ANY OPERATORS

Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root

```
mysql> SELECT CONCAT(first_name, ' ', last_name) AS Actor,
    ->        birth_date AS "Birth Date"
    -> FROM actors;
+--------------------+------------+
| Actor              | Birth Date |
+--------------------+------------+
| Nicholas Cage      | 1964-01-07 |
| Dwayne Johnson     | 1972-05-02 |
| Diane Kruger       | 1976-07-15 |
| Jay Baruchel       | 1982-04-09 |
| Josh Hutcherson    | 1992-10-12 |
| Teresa Palmer      | 1986-02-26 |
| Alexander Ludwig   | 1992-05-07 |
| Vanessa Hudgens    | 1988-12-14 |
| Ryan Sheckler      | 1989-12-30 |
| Jennifer Lawrence  | 1990-08-15 |
| Ben Stiller        | 1965-11-30 |
| AnnaSophia Robb    | 1993-12-08 |
| Dan Stevens        | 1982-10-10 |
+--------------------+------------+
13 rows in set (0.00 sec)

mysql> _
```

< ANY → < MAXIMUM

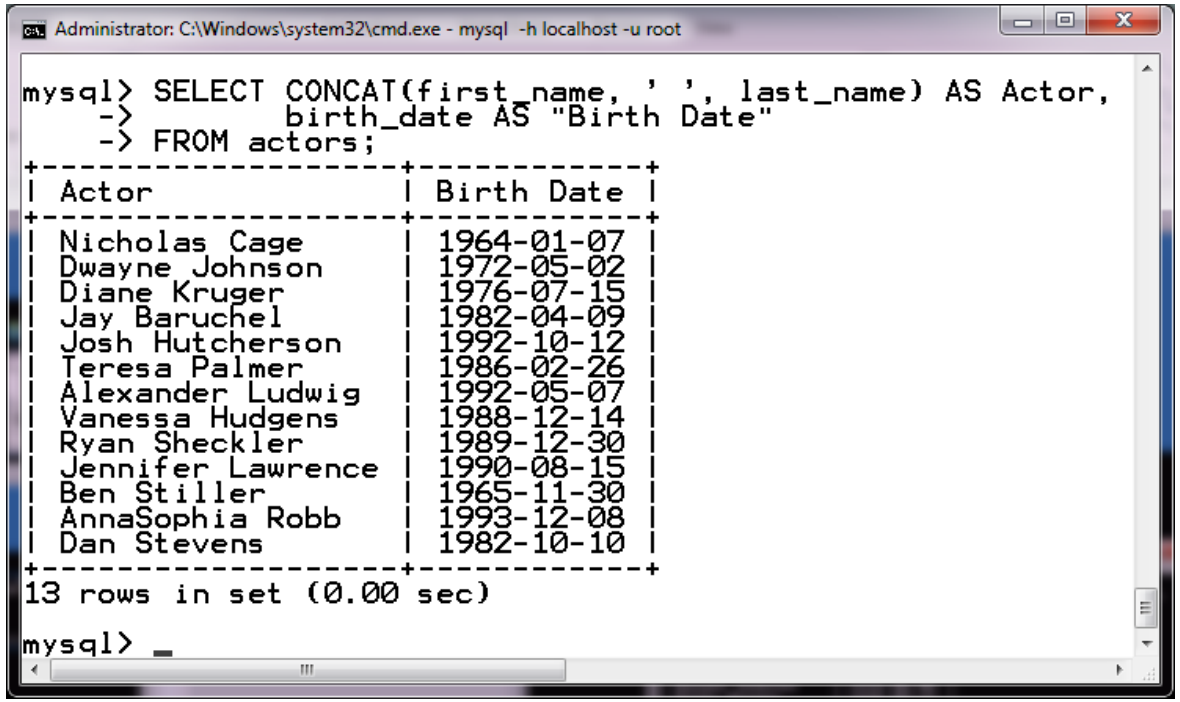
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root

```
mysql> SELECT CONCAT(first_name, ' ', last_name) AS Actor,
    ->        birth_date AS "Birth Date"
    -> FROM actors
    -> WHERE birth_date < ANY
    ->       (SELECT birth_date
    ->        FROM actors
    ->        WHERE YEAR(birth_date) % 2 <> 0);
+--------------------+------------+
| Actor              | Birth Date |
+--------------------+------------+
| Nicholas Cage      | 1964-01-07 |
| Dwayne Johnson     | 1972-05-02 |
| Diane Kruger       | 1976-07-15 |
| Jay Baruchel       | 1982-04-09 |
| Josh Hutcherson    | 1992-10-12 |
| Teresa Palmer      | 1986-02-26 |
| Alexander Ludwig   | 1992-05-07 |
| Vanessa Hudgens    | 1988-12-14 |
| Ryan Sheckler      | 1989-12-30 |
| Jennifer Lawrence  | 1990-08-15 |
| Ben Stiller        | 1965-11-30 |
| Dan Stevens        | 1982-10-10 |
+--------------------+------------+
12 rows in set (0.00 sec)

mysql> _
```

SUBQUERY:ALL & ANY OPERATORS

```
SELECT CONCAT(first_name, ‘ ’, last_name) AS Actor,
       birth_date AS “Birth Date”
FROM actors
WHERE birth_date > ANY
      (SELECT birth_date
      FROM actors
       WHERE YEAR(birth_date) % 2 <> 0);
```



SUBQUERY:ALL & ANY OPERATORS

```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT CONCAT(first_name, ' ', last_name) AS Actor,
    ->        birth_date AS "Birth Date"
    -> FROM actors;
+--------------------+------------+
| Actor              | Birth Date |
+--------------------+------------+
| Nicholas Cage      | 1964-01-07 |
| Dwayne Johnson     | 1972-05-02 |
| Diane Kruger       | 1976-07-15 |
| Jay Baruchel       | 1982-04-09 |
| Josh Hutcherson    | 1992-10-12 |
| Teresa Palmer      | 1986-02-26 |
| Alexander Ludwig   | 1992-05-07 |
| Vanessa Hudgens    | 1988-12-14 |
| Ryan Sheckler      | 1989-12-30 |
| Jennifer Lawrence  | 1990-08-15 |
| Ben Stiller        | 1965-11-30 |
| AnnaSophia Robb    | 1993-12-08 |
| Dan Stevens        | 1982-10-10 |
+--------------------+------------+
13 rows in set (0.00 sec)

mysql> _
```

>ANY → >MINIMUM

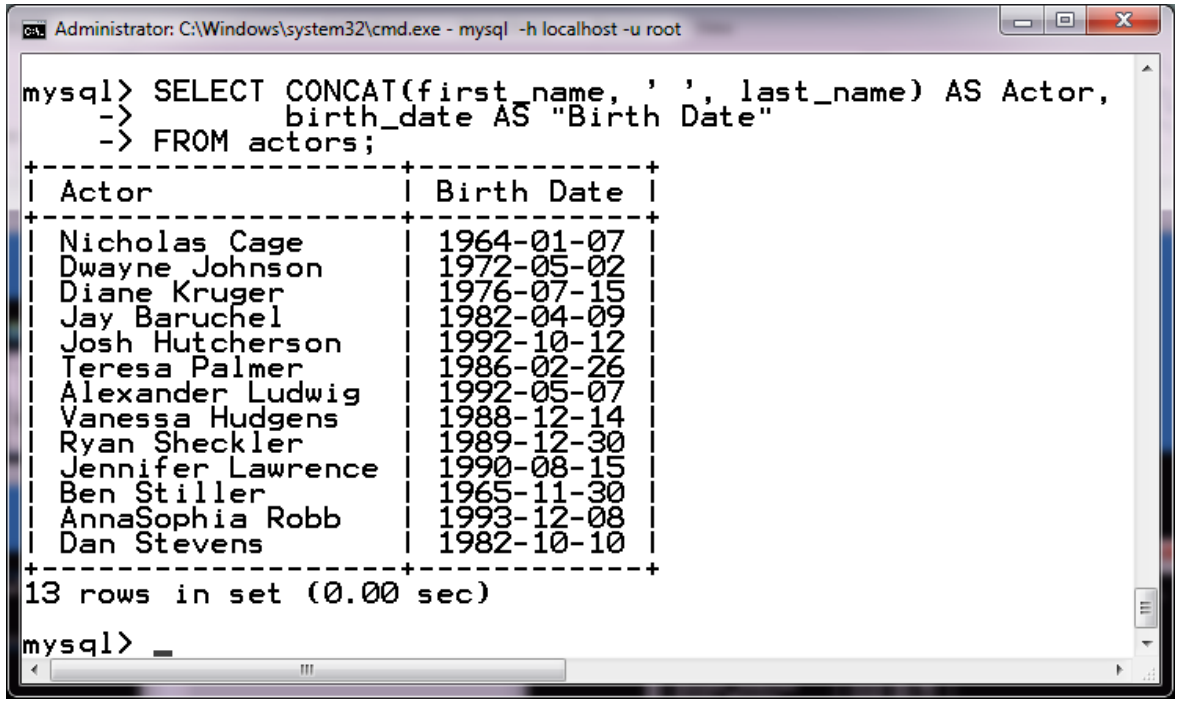
```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT CONCAT(first_name, ' ', last_name) AS Actor,
    ->        birth_date AS "Birth Date"
    -> FROM actors
    -> WHERE birth_date > ANY
    ->       (SELECT birth_date
    ->        FROM actors
    ->        WHERE YEAR(birth_date) % 2 <> 0);
+--------------------+------------+
| Actor              | Birth Date |
+--------------------+------------+
| Dwayne Johnson     | 1972-05-02 |
| Diane Kruger       | 1976-07-15 |
| Jay Baruchel       | 1982-04-09 |
| Josh Hutcherson    | 1992-10-12 |
| Teresa Palmer      | 1986-02-26 |
| Alexander Ludwig   | 1992-05-07 |
| Vanessa Hudgens    | 1988-12-14 |
| Ryan Sheckler      | 1989-12-30 |
| Jennifer Lawrence  | 1990-08-15 |
| AnnaSophia Robb    | 1993-12-08 |
| Dan Stevens        | 1982-10-10 |
+--------------------+------------+
11 rows in set (0.00 sec)

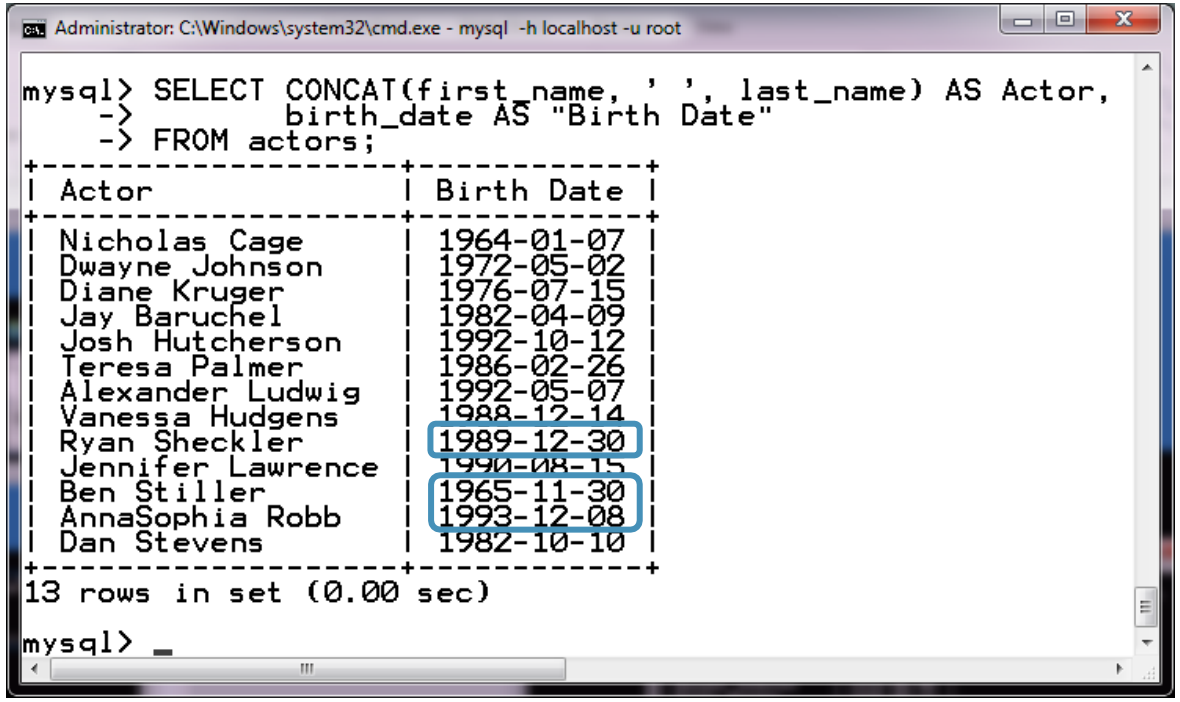
mysql> _
```

SUBQUERY:ALL & ANY OPERATORS

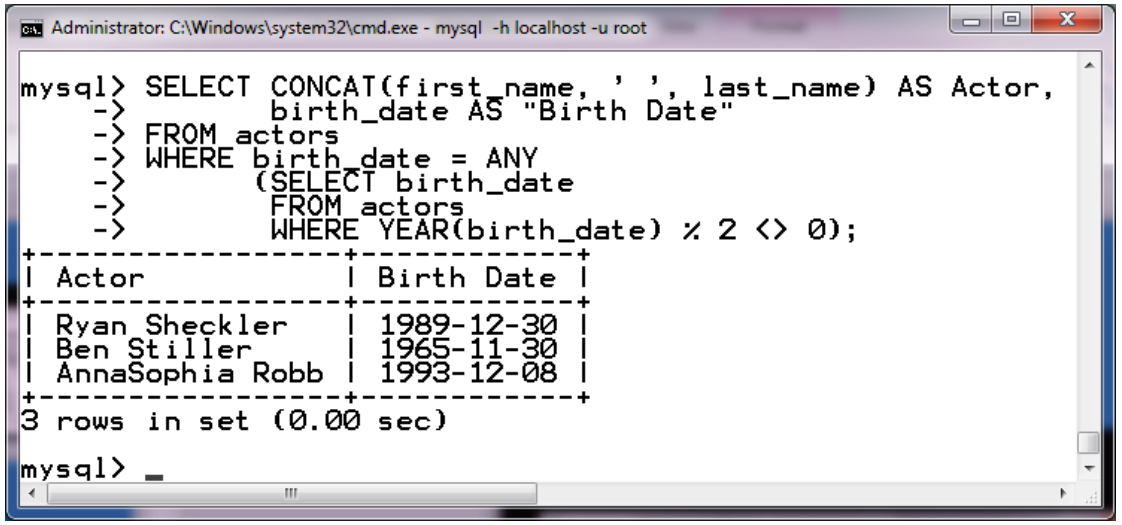
```
SELECT CONCAT(first_name, ‘ ’, last_name) AS Actor,
       birth_date AS “Birth Date”
FROM actors
WHERE birth_date = ANY
      (SELECT birth_date
      FROM actors
       WHERE YEAR(birth_date) % 2 <> 0);
```



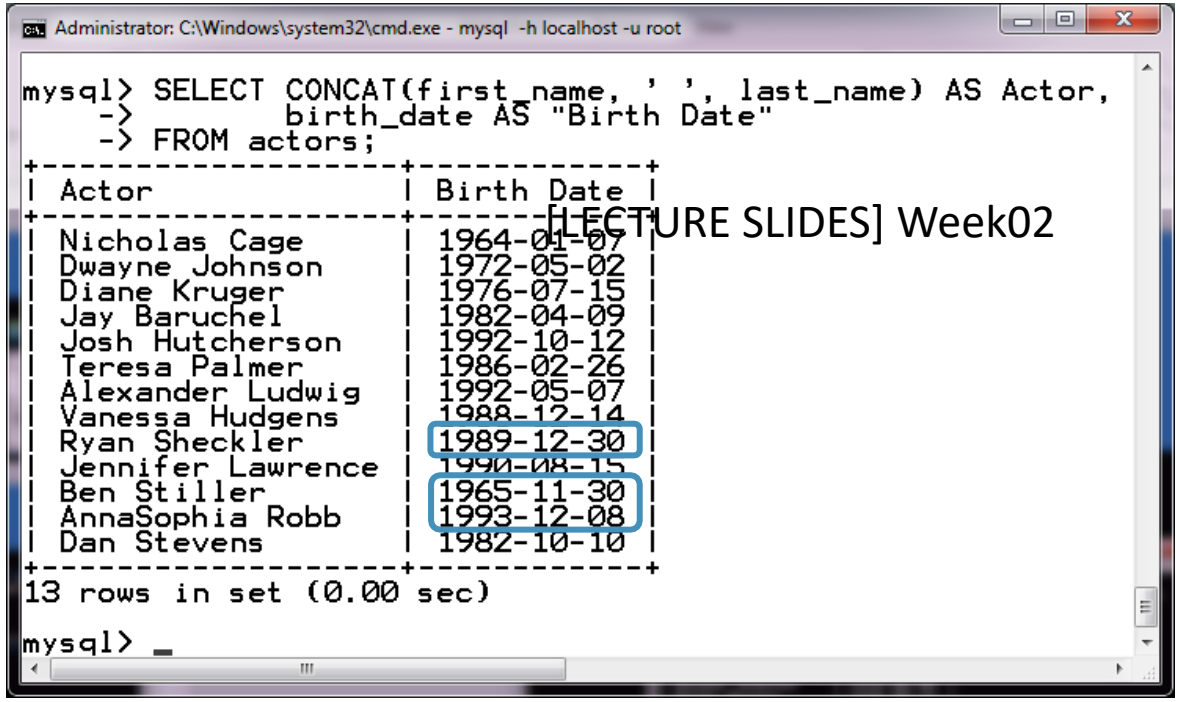
SUBQUERY:ALL &ANY OPERATORS



=ANY → ?

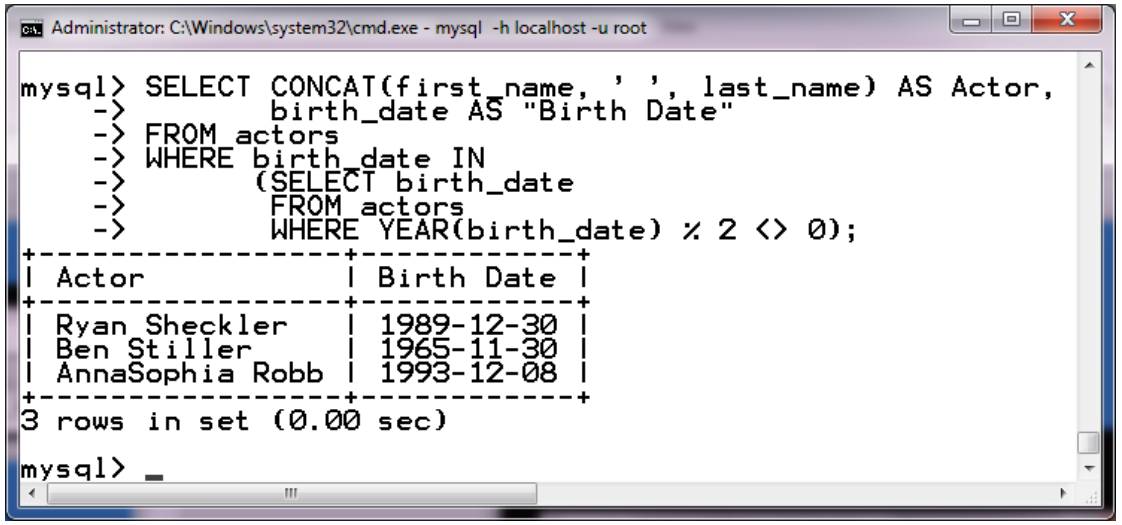


SUBQUERY:ALL & ANY OPERATORS



[LECTURE SLIDES] Week02

=ANY → IN



DENNIS GUNAWAN



IF351 DATABASE SYSTEMS

07 SELECT STATEMENT 3

OUTLINE

JOIN

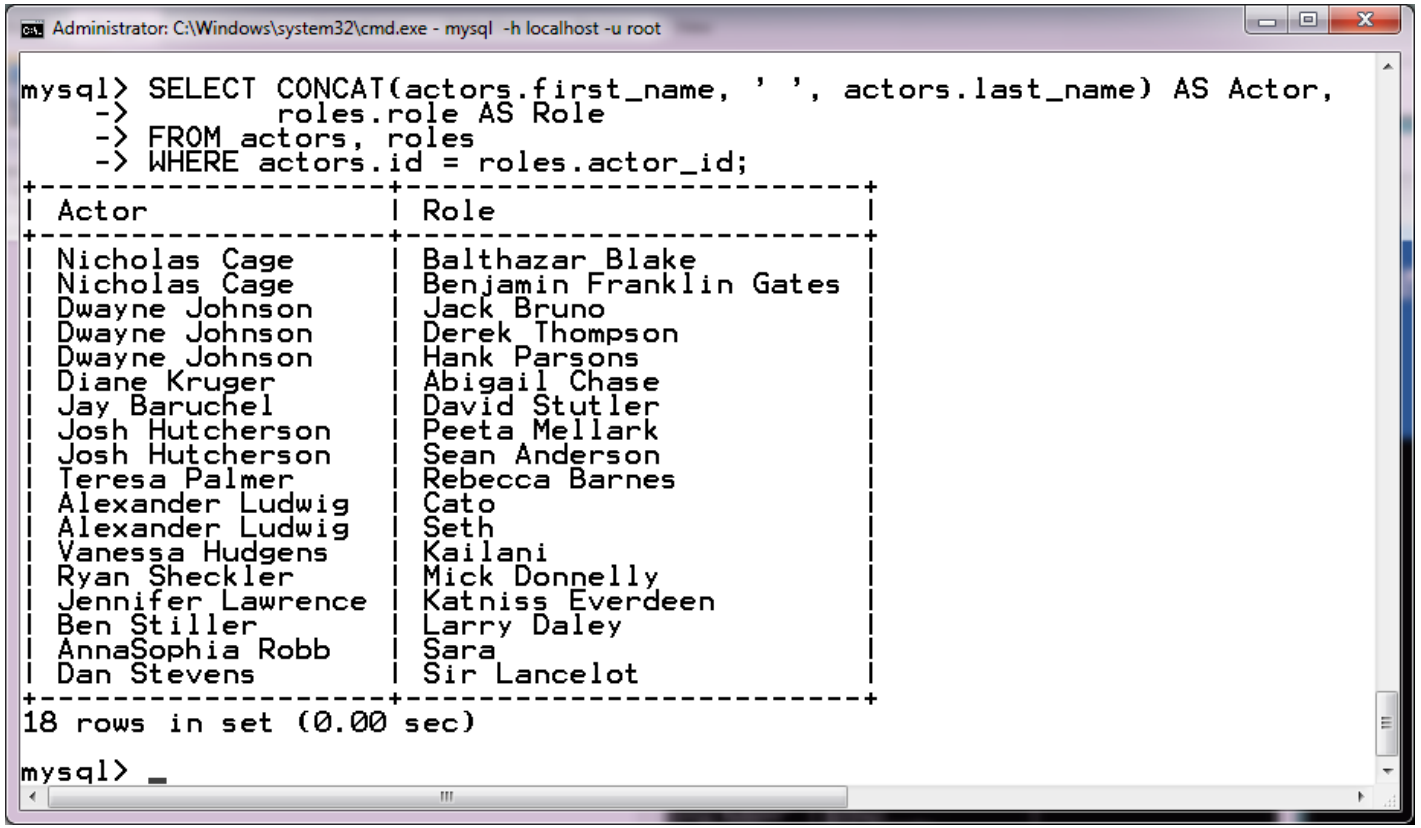
Set Operator

MULTIPLE TABLE SPECIFICATIONS IN THE FROM CLAUSE

```
SELECT CONCAT(first_name, ‘ ’, last_name) AS Actor,
       role AS Role
FROM actors, roles
WHERE id = actor_id;
```

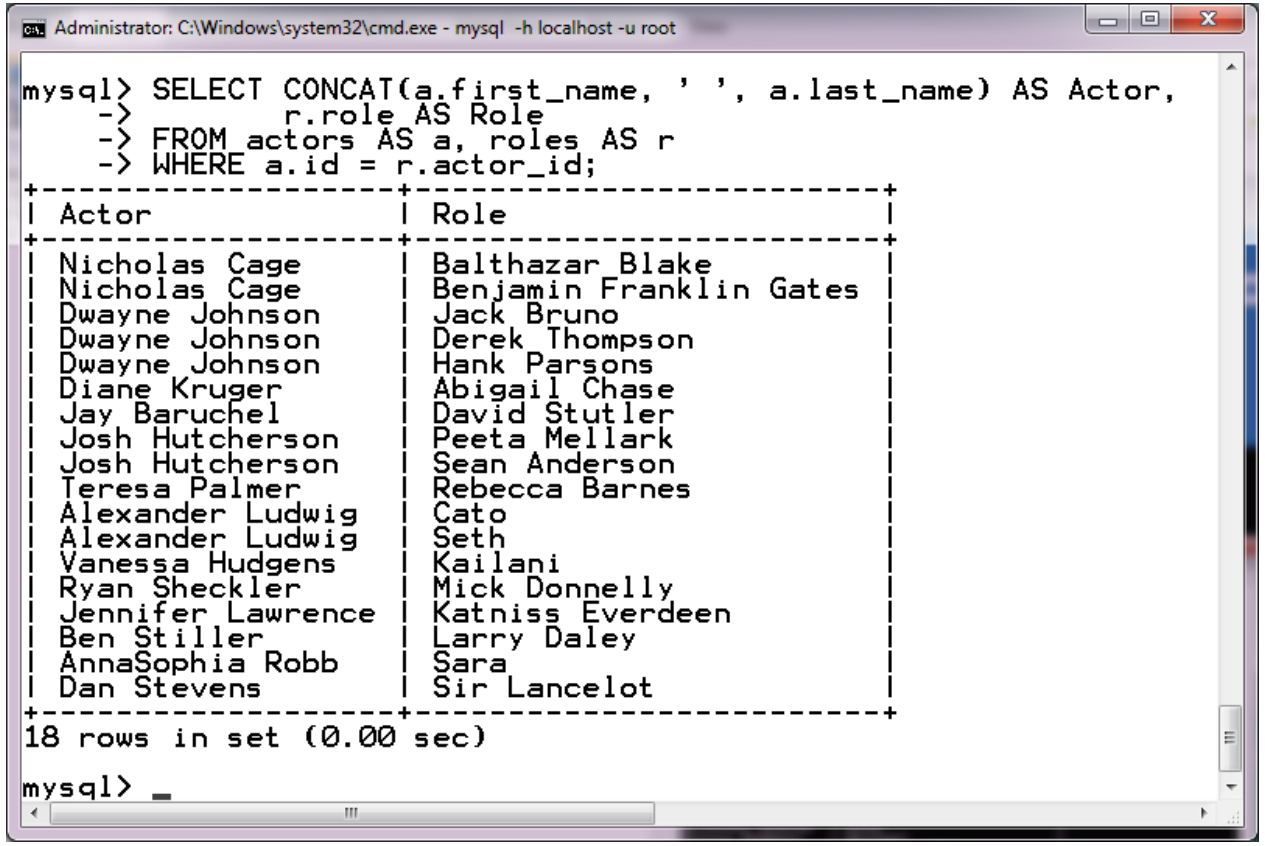
MULTIPLE TABLE SPECIFICATIONS IN THE FROM CLAUSE

```
SELECT CONCAT(actors.first_name, ' ', actors.last_name) AS Actor,
       roles.role AS Role
FROM actors, roles
WHERE actors.id = roles.actor_id;
```



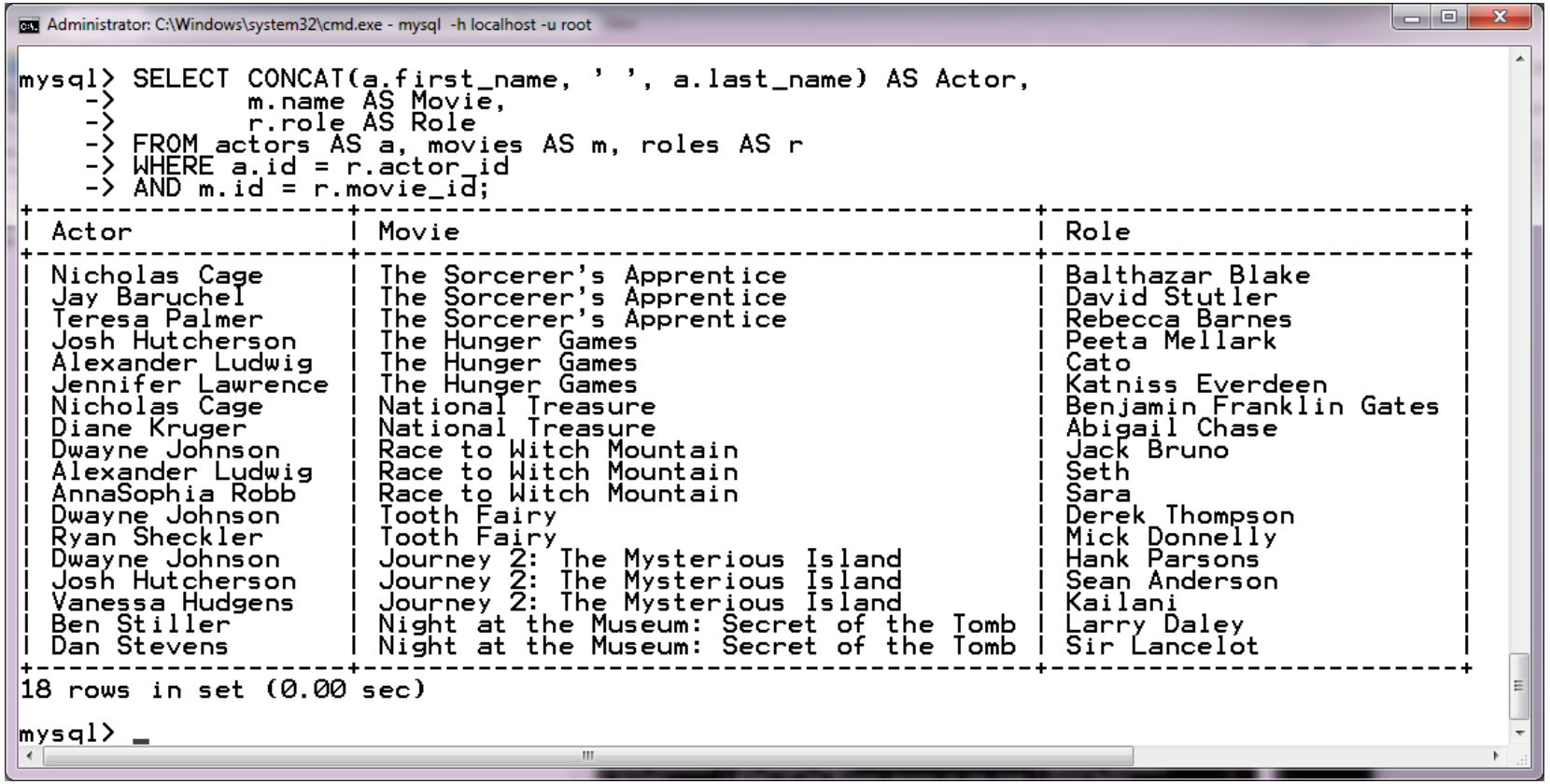
MULTIPLE TABLE SPECIFICATIONS IN THE FROM CLAUSE: PSEUDONYMS FOR TABLE NAMES

```
SELECT CONCAT(a.first_name, ‘ ’, a.last_name) AS Actor,
       r.role AS Role
FROM actors AS a, roles AS r
WHERE a.id = r.actor_id;
```



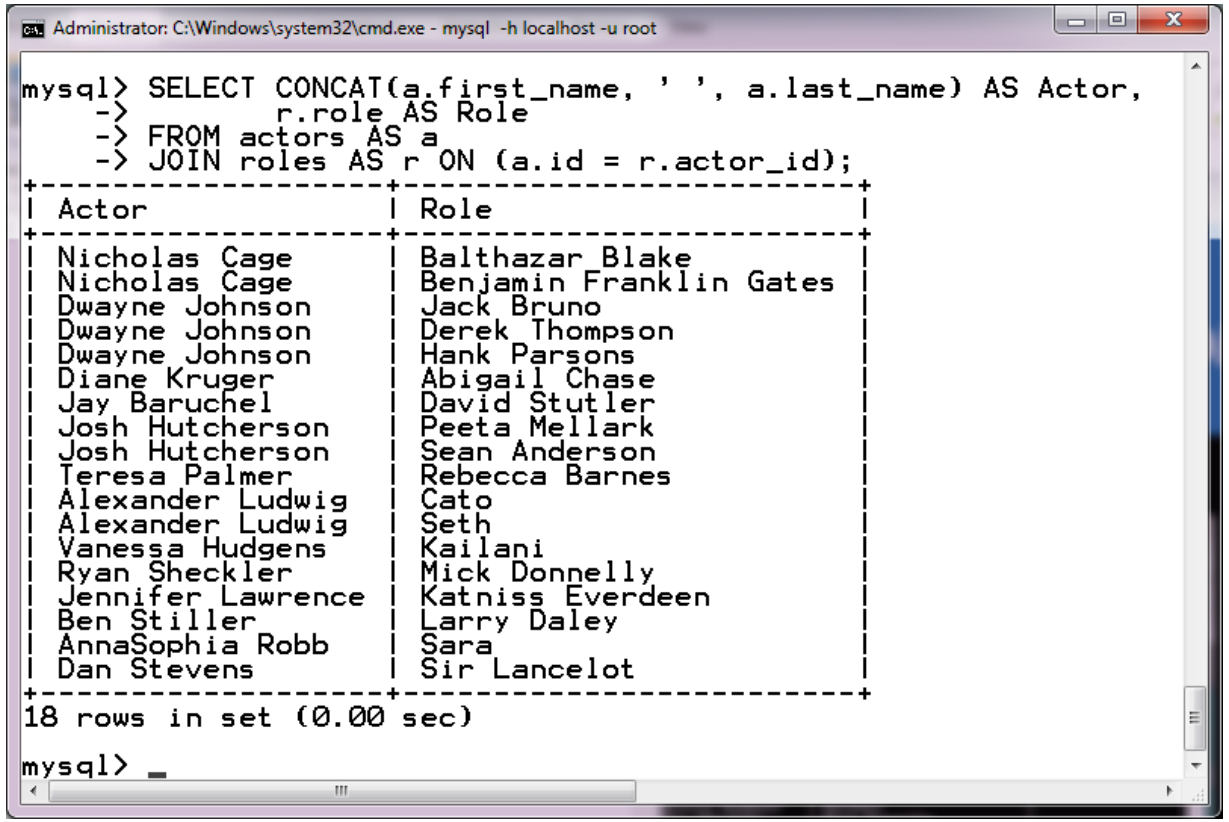
MULTIPLE TABLE SPECIFICATIONS IN THE FROM CLAUSE

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       m.name AS Movie,
       r.role AS Role
FROM actors AS a,
     movies AS m,
     roles AS r
WHERE a.id = r.actor_id
AND m.id = r.movie_id;
```



EXPLICIT JOINS IN THE FROM CLAUSE

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       r.role AS Role
FROM actors AS a
JOIN roles AS r ON (a.id = r.actor_id);
```

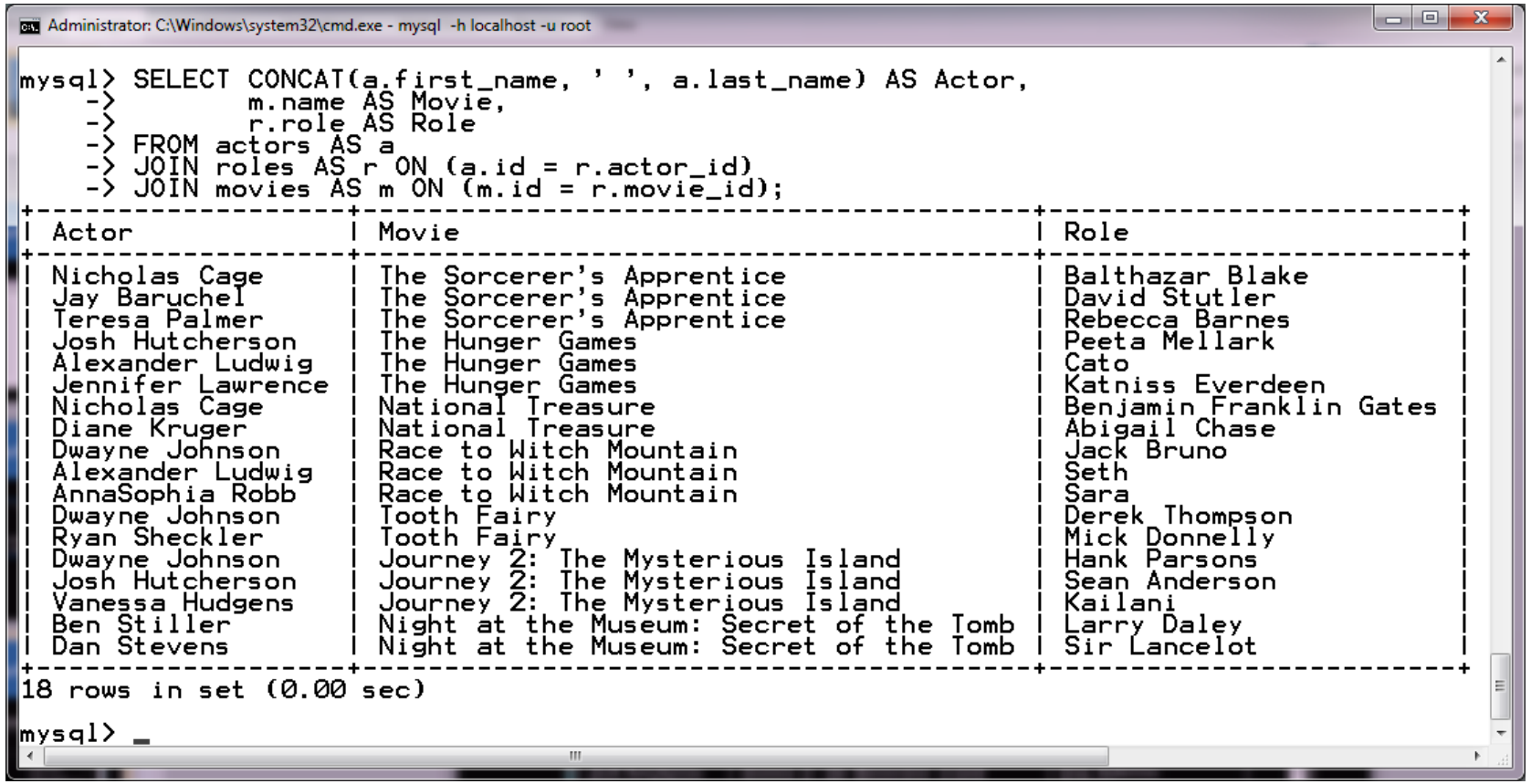


EXPLICIT JOINS IN THE FROM CLAUSE

```
SELECT CONCAT(a.first_name, ‘ ’, a.last_name) AS Actor,
       r.role AS Role
FROM actors AS a
INNER JOIN roles AS r ON (a.id = r.actor_id);
```

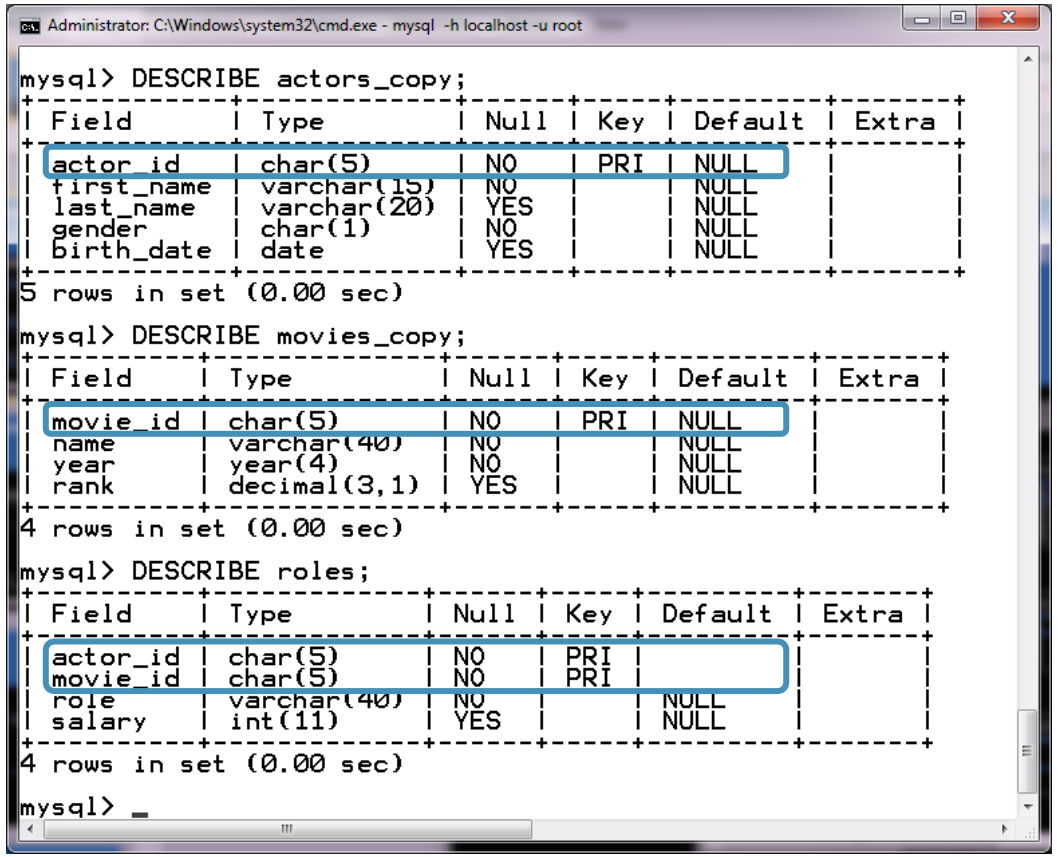
EXPLICIT JOINS IN THE FROM CLAUSE

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       m.name AS Movie,
       r.role AS Role
FROM actors AS a
JOIN roles AS r
ON (a.id = r.actor_id)
JOIN movies AS m
ON (m.id = r.movie_id);
```

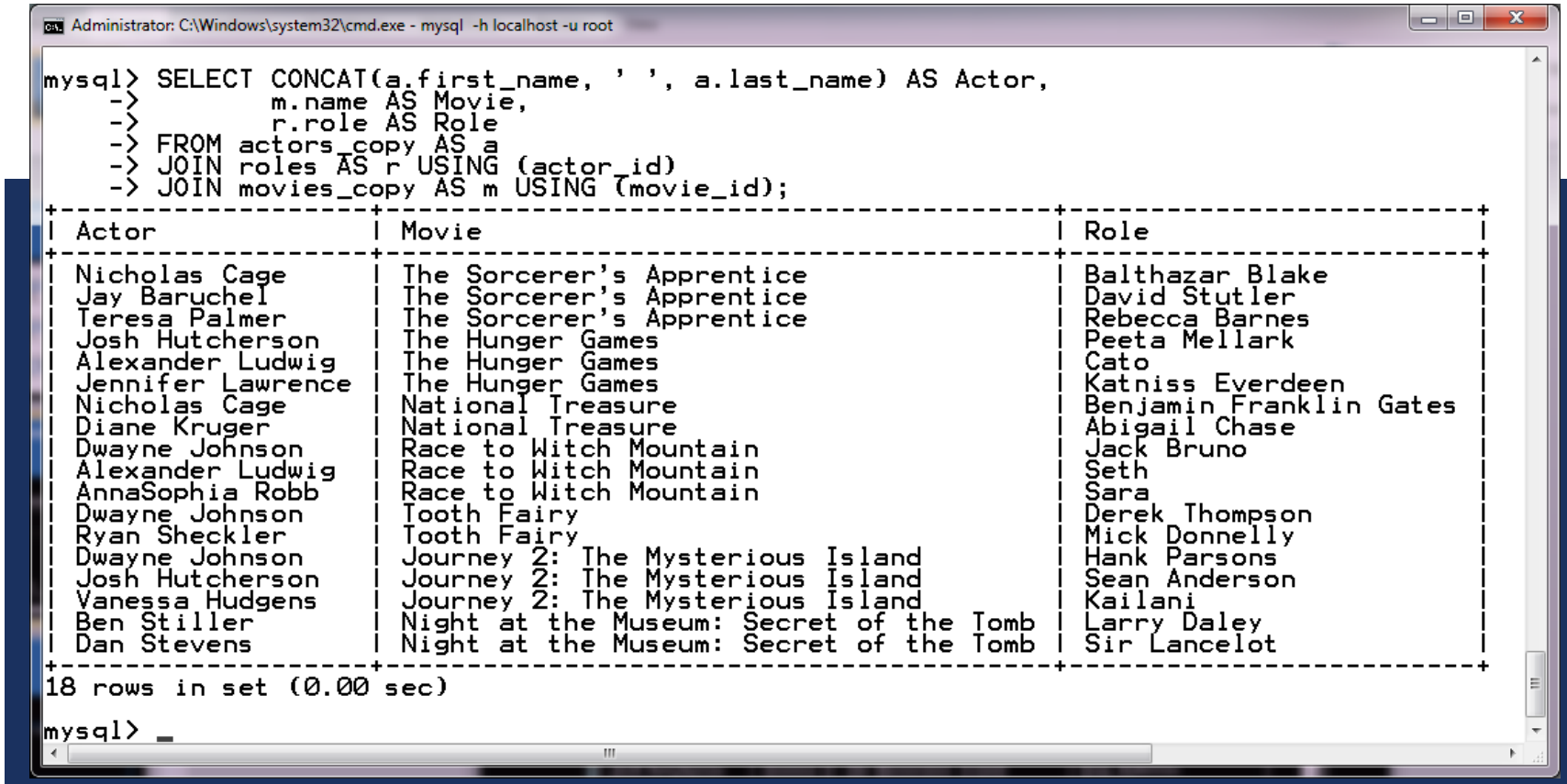


REPLACING JOIN CONDITIONS WITH USING

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       m.name AS Movie,
       r.role AS Role
FROM actors_copy AS a
JOIN roles AS r USING (actor_id)
JOIN movies_copy AS m USING (movie_id);
```

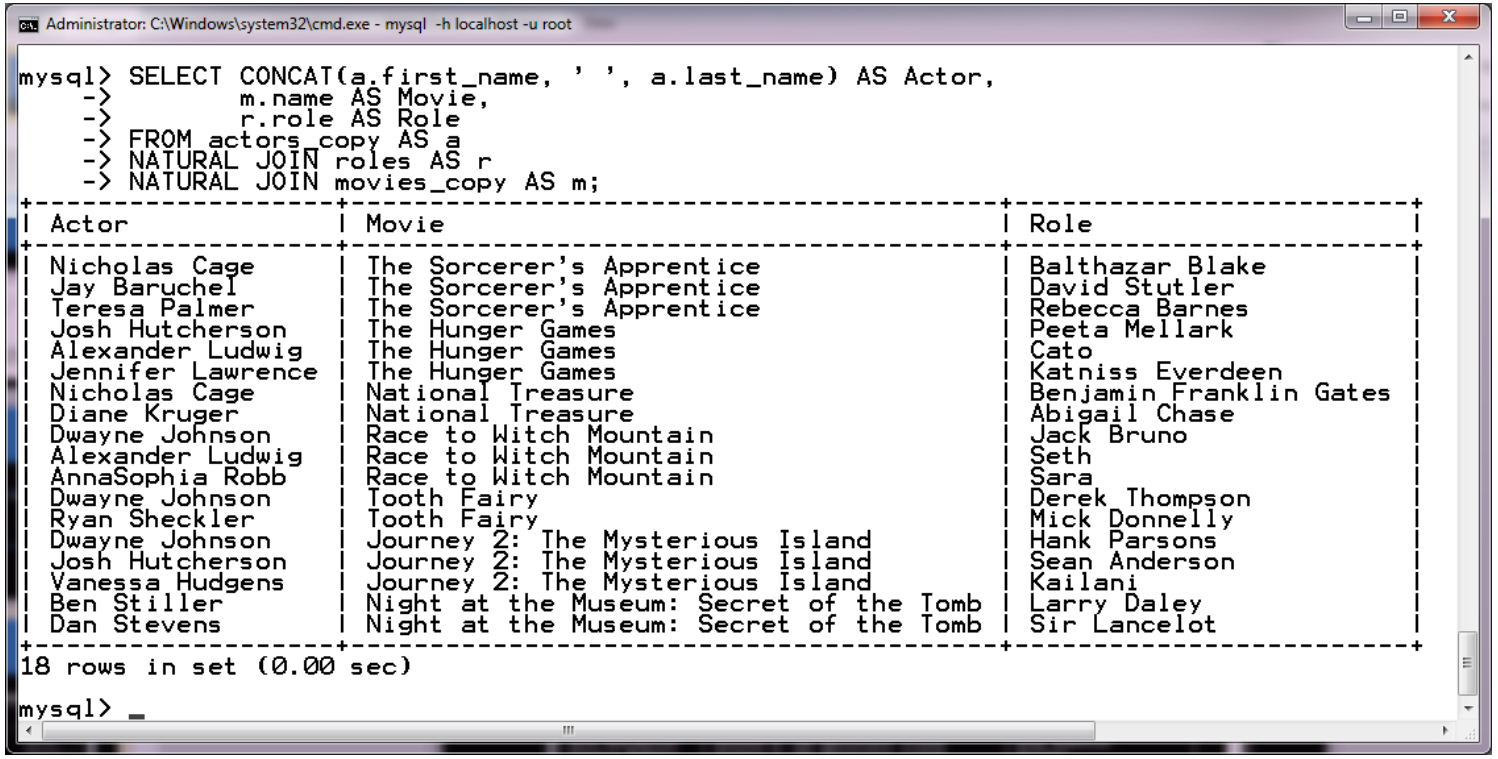


REPLACING JOIN CONDITIONS WITH USING



NATURAL JOIN

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       m.name AS Movie,
       r.role AS Role
FROM actors_copy AS a
NATURAL JOIN roles AS r
NATURAL JOIN movies_copy AS m;
```

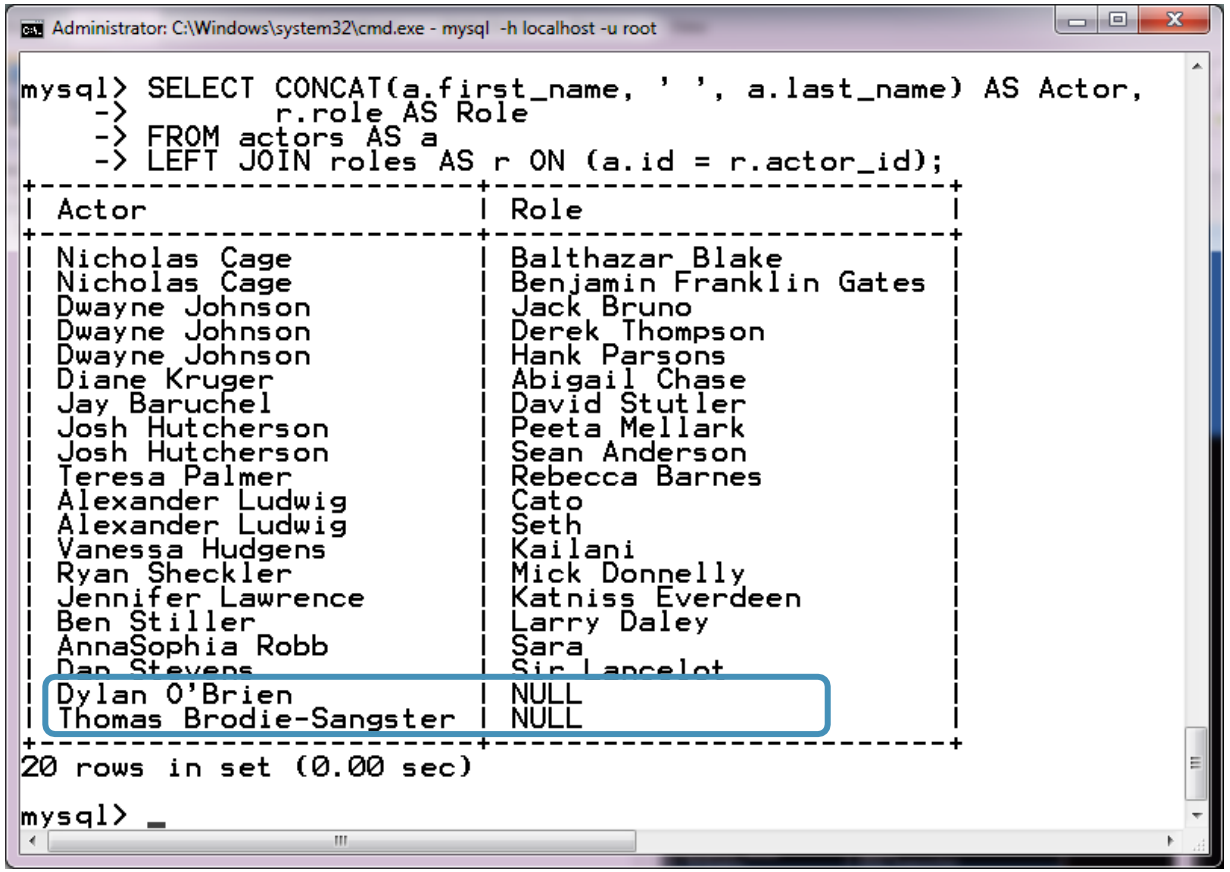



```
mysql> SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
    ->        m.name AS Movie,
    ->        r.role AS Role
    -> FROM actors_copy AS a
    -> NATURAL JOIN roles AS r
    -> NATURAL JOIN movies_copy AS m;
+------------------+-----------------------------------------+-------------------------+
| Actor            | Movie                                   | Role                    |
+------------------+-----------------------------------------+-------------------------+
| Nicholas Cage    | The Sorcerer's Apprentice               | Balthazar Blake         |
| Jay Baruchel     | The Sorcerer's Apprentice               | David Stutler           |
| Teresa Palmer    | The Sorcerer's Apprentice               | Rebecca Barnes          |
| Josh Hutcherson  | The Hunger Games                        | Peeta Mellark           |
| Alexander Ludwig | The Hunger Games                        | Cato                    |
| Jennifer Lawrence| The Hunger Games                        | Katniss Everdeen        |
| Nicholas Cage    | National Treasure                       | Benjamin Franklin Gates |
| Diane Kruger     | National Treasure                       | Abigail Chase           |
| Dwayne Johnson   | Race to Witch Mountain                  | Jack Bruno              |
| Alexander Ludwig | Race to Witch Mountain                  | Seth                    |
| AnnaSophia Robb  | Race to Witch Mountain                  | Sara                    |
| Dwayne Johnson   | Tooth Fairy                             | Derek Thompson          |
| Ryan Sheckler    | Tooth Fairy                             | Mick Donnelly           |
| Dwayne Johnson   | Journey 2: The Mysterious Island        | Hank Parsons            |
| Josh Hutcherson  | Journey 2: The Mysterious Island        | Sean Anderson           |
| Vanessa Hudgens  | Journey 2: The Mysterious Island        | Kailani                 |
| Ben Stiller      | Night at the Museum: Secret of the Tomb | Larry Daley             |
| Dan Stevens      | Night at the Museum: Secret of the Tomb | Sir Lancelot            |
+------------------+-----------------------------------------+-------------------------+
18 rows in set (0.00 sec)

mysql> _
```

OUTER JOINS: LEFT OUTER JOIN

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       r.role AS Role
FROM actors  AS  a
LEFT JOIN roles AS r
ON (a.id = r.actor_id);
```

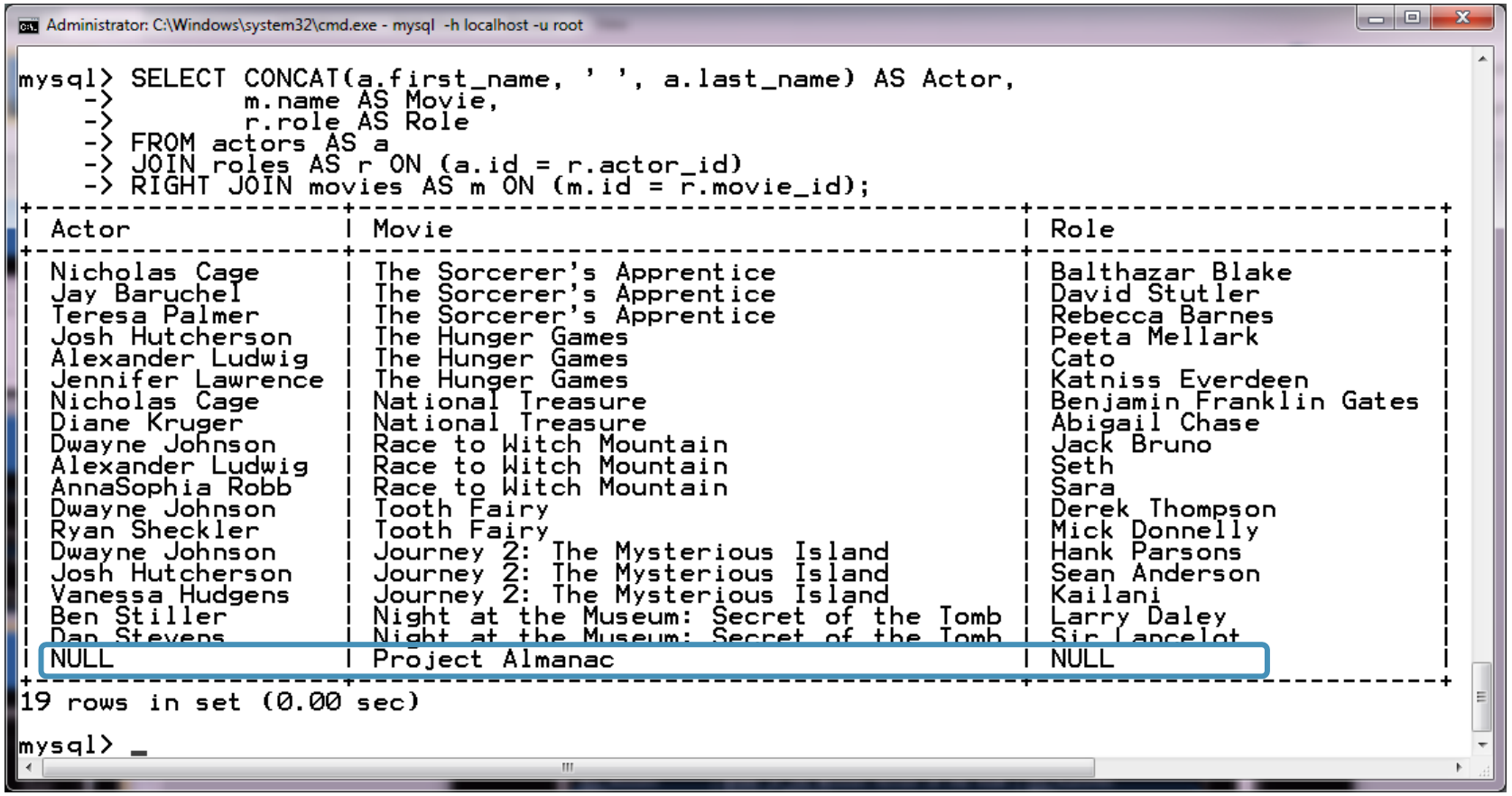


OUTER JOINS: LEFT OUTER JOIN

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       r.role AS Role
FROM actors AS a
LEFT OUTER JOIN roles AS r ON (a.id = r.actor_id);
```

OUTER JOINS: RIGHT OUTER JOIN

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       m.name AS Movie,
       r.role AS Role
FROM actors AS a
JOIN roles AS r
ON (a.id = r.actor_id)
RIGHT JOIN movies AS m
ON (m.id = r.movie_id);
```

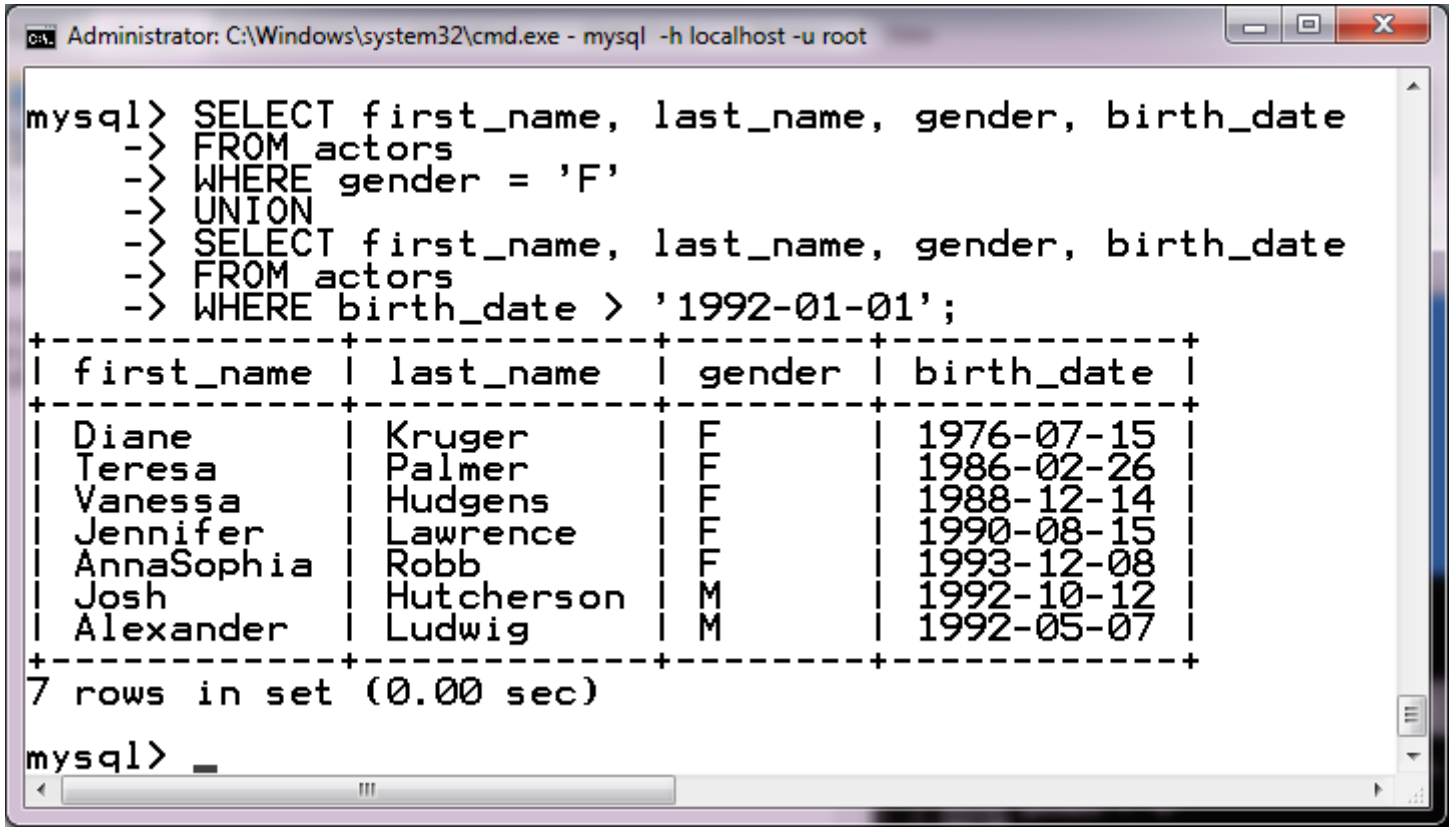


OUTER JOINS: RIGHT OUTER JOIN

```
SELECT CONCAT(a.first_name, ' ', a.last_name) AS Actor,
       m.name AS Movie,
       r.role AS Role
FROM actors AS a
JOIN roles AS r ON (a.id = r.actor_id)
RIGHT OUTER JOIN movies AS m ON (m.id = r.movie_id);
```

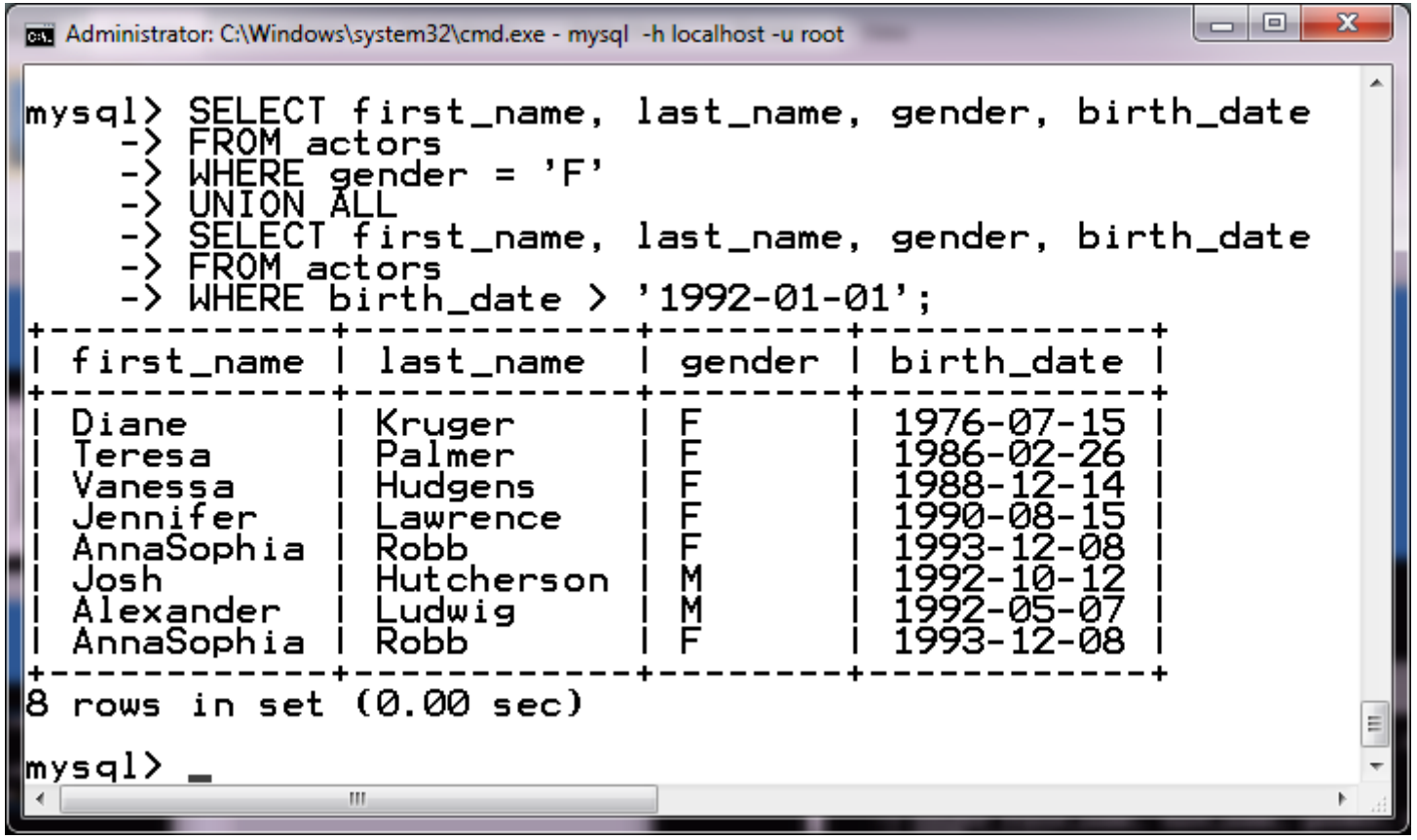
SET OPERATOR: UNION

```
SELECT first_name, last_name,
       gender, birth_date
FROM actors
WHERE gender = 'F'
UNION
SELECT first_name, last_name,
       gender, birth_date
FROM actors
WHERE birth_date > '1992-01-01';
```



SET OPERATOR: UNION ALL

```
SELECT first_name, last_name,
       gender, birth_date
FROM actors
WHERE gender = 'F'
UNION ALL
SELECT first_name, last_name,
       gender, birth_date
FROM actors
WHERE birth_date > '1992-01-01';
```



REFERENCES

Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley.

Rick F. van der Lans (2007), SQL for MySQL Developers, Addison Wesley.

Connolly, Thomas and Carolyn Begg (2015), Database Systems A Practical Approach to Design, Implementation, and Management, 6th edition, Pearson Education, ISBN 13: 978-1-292-06118-4, Essex, England.

```
SELECT *
FROM WORLD
WHERE SOMEONE
LIKE '%YOU%';

EMPTY SET (0.00 SEC)
```

Vision

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.

INFORMATIKA
UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Mission

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology